Análisis algorítmico de ALNS y ALNS con Q-learning para el VRPTW

Estructura de datos, métodos de selección, operadores y optimizaciones

Documento técnico de apoyo a la tesis

31 de octubre de 2026

Resumen

Este documento describe, a nivel de pseudocódigo y con su justificación teórica, la implementación de los dos métodos comparados en la tesis para el problema de ruteo de vehículos con ventanas de tiempo (VRPTW): un ALNS con selección adaptativa por ruleta (`ALNS_Classic`) y un ALNS cuya selección de operadores se aprende con Q-learning (`ALNS_QLearning`). Ambos comparten el mismo bucle de búsqueda en dos fases (minimización de rutas y minimización de distancia) y el mismo ritmo de aprendizaje (eligen en cada iteración y actualizan sus preferencias cada 100 iteraciones); solo difieren en cómo eligen los operadores y en qué premian. Se detallan el bucle común, los dos mecanismos de selección y aprendizaje, los seis operadores de destrucción, los cinco de reparación y las optimizaciones algorítmicas que determinan el costo por iteración. Todos los valores numéricos corresponden a las constantes y parámetros por defecto del código fuente. El documento se cierra con los resultados de ambos métodos en instancias de 100 (Solomon) y de 200, 400, 600 y 800 clientes (Gehring y Homberger): empatan en 100 clientes y, desde 200, el Q-learning obtiene con la misma flota rutas más cortas, con una ventaja que crece con el tamaño (de 0.34 % a 1.50 % de distancia media) y se concentra en las clases de rutas cortas.

Índice

1. Alcance y organización del código **4**

2. Modelo de datos y evaluación de factibilidad **4**
2.1. Instancia 4
2.2. Ruta 5
2.3. Evaluación de una inserción en tiempo constante 6
2.4. Solución y función de costo 6
2.5. Solución inicial 7

3. Bucle de búsqueda común **7**
3.1. Idea general 7
3.2. Bucle principal 8
3.3. Las dos fases de la búsqueda 9
3.4. Ganancias relativas 10
3.5. Criterio de aceptación 11
3.6. Esquema de temperatura 11
3.7. Catálogo de operadores y grado de destrucción 12

4. ALNS clásico: selección por ruleta adaptativa **13**
4.1. Selección 13
4.2. Asignación de crédito y actualización 13
4.3. Propiedades relevantes 14

5. ALNS con Q-learning **15**
5.1. La idea en pocas palabras . . . 15
5.2. Estados, acciones y tablas . . . 16
5.3. Selección: política ε-voraz por fases . . . 16
5.4. Recompensa: mejora relativa normalizada . . . 17
5.5. Actualización de la tabla al cerrar el segmento . . . 18
5.6. Cambio de fase . . . 18
5.7. Propiedades relevantes . . . 19

6. Operadores de destrucción **19**
6.1. Extracción aleatoria . . . 20
6.2. Extracción de rutas . . . 20
6.3. Extracción de los peores . . . 20
6.4. Extracción de Shaw . . . 21
6.5. Extracción de grupos . . . 22
6.6. Extracción por ventana de tiempo . . . 23

7. Operadores de reparación **23**
7.1. Formulación directa y su costo . . . 24
7.2. Inserción con caché por ruta . . . 24
7.3. Ruido en los costos de inserción . . . 25
7.4. Agregadores y actualización perezosa . . . 25
7.5. Resumen de las optimizaciones . . . 27

8. Síntesis: características que determinan el comportamiento **27**
8.1. Características comunes . . . 27
8.2. Diferencias entre los dos métodos . . . 28

9. Resultados experimentales **29**
9.1. Diseño experimental . . . 29
9.2. Métricas . . . 30
9.3. Resumen por tamaño . . . 30
9.4. Resultados por tamaño . . . 31
9.4.1. 100 clientes (Solomon) . . . 31
9.4.2. 200 clientes . . . 32
9.4.3. 400 clientes . . . 32
9.4.4. 600 clientes . . . 33
9.4.5. 800 clientes . . . 33
9.5. Dónde surge la ventaja de la propuesta . . . 34

Índice de algoritmos

1. RecalcularRuta (`Route::recalculate`) . . . 5
2. EvaluarInserción (`evalInsertion`) . . . 6
3. SoluciónInicial (`Solution::generateInitialSolution`) . . . 7
4. Bucle ALNS común en dos fases (`ALNS::solve`) . . . 8
5. IniciarEtapa (función local `startStage` de `ALNS::solve`) . . . 9
6. AplicarOperadores (`applyOperators`) . . . 12
7. Selección y aprendizaje del ALNS clásico (`ALNS_Classic` y `Roulette`) . . . 14
8. Selección ε-voraz (`ALNS_QLearning::selectOperators` y `QTable::select`) . . . 17
9. Aprendizaje por Q-learning (`learn` y `QTable::closeSegment`) . . . 18
10. Inicio de la fase 2 en el Q-learning (`ALNS_QLearning::onDistancePhase`) . . . 18

11. ExtracciónAleatoria (`randomRemoval`) 20
12. ExtracciónDeRutas (`routeRemoval`) 20
13. ExtracciónDePeores (`worstRemoval`) 21
14. ExtracciónDeShaw (`shawRemoval`) 22
15. ExtracciónDeGrupos (`clusterRemoval`) 22
16. ExtracciónPorVentana (`timeWindowRemoval`) 23
17. InserciónConCaché$\langle G \rangle$ (`cachedInsertion<Agg>`) 24
18. Entrada (`computeEntry`) 25

1. Alcance y organización del código

El solver está escrito en C++ y separa con claridad tres responsabilidades: la representación del problema, los operadores de vecindario y el control de la búsqueda. La Tabla 1 resume qué contiene cada módulo y en qué sección de este documento se analiza.

Tabla 1: Módulos del solver y sección donde se documentan.

Archivo	Contenido	Sección
`VRPTW Environment/instance.*`	Instancia, cliente, ruta; matrices precalculadas; recálculo de una ruta	2
`VRPTW Environment/solution.*`	Solución, función de costo, solución inicial	2
`Utils/params.h`	Parámetros ajustables del Q-learning (`SolverParams`)	5
`ALNS/alns.*`	Bucle común en dos fases (rutas y distancia): recocido simulado, aceptación, reinicio por estancamiento; ganancias relativas; ruleta adaptativa (`Roulette`)	3
`ALNS/alns_classic.*`	Selección por dos ruletas (destrucción y reparación)	4
`ALNS/alns_qlearning.*`	Selección ε-voraz con dos tablas Q (destrucción y reparación) que aprenden por segmentos	5
`Operators/operators.*`	Catálogo de operadores y grado de destrucción	3
`Operators/destroy_ops.cpp`	Seis operadores de destrucción	6
`Operators/repair_ops.cpp`	Evaluación de inserción y cinco operadores de reparación	7

La clase abstracta `ALNS` implementa el bucle de búsqueda completo y delega únicamente dos decisiones en sus subclases, mediante los métodos virtuales `selectOperators` y `learn`. Un tercer método virtual, `onDistancePhase`, es solo un aviso de que comienza la segunda fase de la búsqueda (Sección 3.3); por defecto no hace nada. En consecuencia, `ALNS_Classic` y `ALNS_QLearning` comparten *exactamente* la misma solución inicial, los mismos operadores, el mismo criterio de aceptación, el mismo esquema de temperatura, la misma división en fases y el mismo generador de números aleatorios (`std::mt19937`, sembrado con la misma semilla para ambos algoritmos en cada corrida). Lo único que cambia entre los dos métodos es *qué par de operadores se elige en cada iteración y cómo se actualiza esa preferencia*. Esta propiedad de diseño es la que permite atribuir las diferencias de resultados al mecanismo de selección y no a otro componente.

Notación. Se usa n para el número de clientes, m para el número de rutas de una solución, $\bar{L} = n/m$ para la longitud media de una ruta, q para el número de clientes que se extraen en una iteración (grado de destrucción) y $T_{\text{máx}}$ para el presupuesto de iteraciones que recibe el solver. $T_{\text{máx}}$ es el número exacto de iteraciones de la fase de minimización de distancia; la fase previa de minimización de rutas consume, como máximo, otras $T_{\text{máx}}$, de modo que una corrida ejecuta entre $T_{\text{máx}}$ y $2\,T_{\text{máx}}$ iteraciones. En los experimentos de la tesis $T_{\text{máx}} = 25\,000$ y cada instancia se resuelve con 10 semillas por algoritmo.

2. Modelo de datos y evaluación de factibilidad

2.1. Instancia

Una instancia (`Instance`) se lee en el formato de Solomon [8], que también usan las instancias de Gehring y Homberger [1]. Contiene el depósito (nodo 0) y n clientes ($N = n + 1$ nodos). Cada nodo i tiene coordenadas (x_i, y_i), demanda μ_i, ventana de tiempo $[a_i, b_i]$ (`ready_time`, `due_date`) y tiempo de servicio s_i. Todos los vehículos tienen capacidad Q. El archivo declara además un tamaño de flota, que el solver lee pero *no impone* como restricción: el número de

vehículos es un objetivo, no un límite. La flota con la que trabaja cada etapa de la búsqueda la fija el propio algoritmo (Sección 3.3).

Al construir la instancia se precalculan dos matrices $N \times N$ que se consultan en tiempo constante durante toda la búsqueda:

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}, \tag{1}$$

$$\mathrm{alc}(i,j) = \big[\, i \neq j \ \wedge\ a_i + s_i + d_{ij} \leq b_j \,\big]. \tag{2}$$

La distancia (1) es euclidiana en doble precisión, sin redondeo ni truncamiento, y coincide con el tiempo de viaje. La matriz de alcanzabilidad (2) indica si es posible visitar j inmediatamente después de i en el mejor caso (saliendo de i lo antes posible); si $\mathrm{alc}(i,j)$ es falso, el arco (i,j) no puede aparecer en ninguna ruta factible. Es una condición necesaria, no suficiente, y se usa como filtro previo barato. Durante el mismo precálculo se guarda la mayor distancia entre dos nodos, $d_{\max} = \max_{i,j} d_{ij}$ (`max_dist`), que fija la amplitud del ruido de los operadores de reparación (Sección 7.3).

La instancia guarda además una tercera estructura, `shaw_order`, que se construye de forma perezosa la primera vez que se necesita y se describe en la Sección 6.4.

2.2. Ruta

Una ruta (`Route`) es una secuencia $\pi = (\pi_0, \pi_1, \ldots, \pi_{L-1})$ con $\pi_0 = \pi_{L-1} = 0$. Una ruta con $L = 2$ está vacía y no cuenta como vehículo usado. Junto a la secuencia se almacenan su carga, su distancia y tres vectores de longitud L:

- A_k: instante de llegada a π_k (`arrival_times`);
- $W_k = \max(0,\, a_{\pi_k} - A_k)$: espera en π_k hasta que abre su ventana (`wait_times`);
- F_k: holgura temporal hacia adelante (`time_slacks`), es decir, cuánto puede retrasarse el inicio del servicio en π_k sin violar la ventana de π_k ni la de ningún nodo posterior de la ruta.

El inicio del servicio en π_k es $\max(A_k, a_{\pi_k})$. La holgura sigue la recurrencia clásica de Savelsbergh [6]:

$$F_{L-1} = b_0 - \max(A_{L-1}, a_0), \qquad F_k = \min\big(b_{\pi_k} - \max(A_k, a_{\pi_k}),\ F_{k+1} + W_{k+1}\big). \tag{3}$$

El segundo término expresa que un retraso en π_k se transmite al nodo siguiente, donde primero consume la espera W_{k+1} (tiempo que de todos modos se iba a perder) y solo el excedente consume su holgura F_{k+1}.

El Algoritmo 1 recalcula todos estos valores con una pasada hacia adelante y otra hacia atrás, en tiempo $\mathcal{O}(L)$. Se invoca cada vez que una ruta cambia.

Algoritmo 1: RecalcularRuta (`Route::recalculate`)

Entrada : ruta π de longitud L, instancia
Salida : carga, distancia y vectores A, W, F de la ruta

$carga \leftarrow 0$; $dist \leftarrow 0$; $A_0 \leftarrow 0$; $W_0 \leftarrow 0$
para $k \leftarrow 0$ **hasta** $L-2$ **hacer**
  $u \leftarrow \pi_k$; $v \leftarrow \pi_{k+1}$
  $dist \leftarrow dist + d_{uv}$
  si $u \neq 0$ **entonces** $carga \leftarrow carga + \mu_u$
  $A_{k+1} \leftarrow \max(A_k, a_u) + s_u + d_{uv}$ ▷ *inicio de servicio en u + servicio + viaje*
  $W_{k+1} \leftarrow \max(0,\, a_v - A_{k+1})$
$F_{L-1} \leftarrow b_0 - \max(A_{L-1}, a_0)$
para $k \leftarrow L-2$ **hasta** 0 **hacer**
  $F_k \leftarrow \min\big(b_{\pi_k} - \max(A_k, a_{\pi_k}),\ F_{k+1} + W_{k+1}\big)$

2.3. Evaluación de una inserción en tiempo constante

La operación más frecuente de todo el solver es preguntar si un cliente u puede insertarse entre π_k y π_{k+1} y cuánto aumenta la distancia. Con los vectores A, W y F la respuesta se obtiene en $\mathcal{O}(1)$, sin recorrer el resto de la ruta (Algoritmo 2).

Algoritmo 2: EvaluarInserción (`evalInsertion`)

Entrada : cliente u, ruta π con A, W, F vigentes, posición k
Salida : (*factible*, Δ)

$i \leftarrow \pi_k$; $\quad j \leftarrow \pi_{k+1}$
si $\neg\,\mathrm{alc}(i,u)$ **o** $\neg\,\mathrm{alc}(u,j)$ **entonces devolver** (falso, –)
$A_u \leftarrow \max(A_k, a_i) + s_i + d_{iu}$ ▷ *llegada a u*
si $A_u > b_u$ **entonces devolver** (falso, –)
$A'_j \leftarrow \max(A_u, a_u) + s_u + d_{uj}$ ▷ *nueva llegada a j*
retraso $\leftarrow \max(0,\ A'_j - A_{k+1})$
si *retraso* $> W_{k+1} + F_{k+1}$ **entonces devolver** (falso, –)
$\Delta \leftarrow d_{iu} + d_{uj} - d_{ij}$
devolver (verdadero, Δ)

La corrección de la prueba se sigue de la definición de F: el retraso en la llegada a j es absorbido primero por la espera W_{k+1}, y el inicio del servicio en j se desplaza en $\max(0, \mathit{retraso} - W_{k+1})$, que por (3) es admisible para j y para todos los nodos posteriores si y solo si no supera F_{k+1}. La restricción de capacidad no depende de la posición, por lo que se verifica una sola vez por ruta ($\mathit{carga} + \mu_u \le Q$) antes de recorrer sus posiciones.

Evaluar todas las posiciones de una ruta cuesta entonces $\mathcal{O}(L)$ en lugar de $\mathcal{O}(L^2)$, y evaluar un cliente contra toda la solución cuesta $\mathcal{O}(n+m)$. El precio es que cada modificación de una ruta exige un recálculo $\mathcal{O}(L)$ (Algoritmo 1), que es despreciable frente al número de evaluaciones que habilita.

2.4. Solución y función de costo

Una solución (`Solution`) x contiene el vector de rutas $\mathcal{R}(x)$, la lista $\mathcal{U}(x)$ de clientes sin asignar (`unassigned`) y dos métricas agregadas que mantiene `updateMetrics`: el número de vehículos $K(x)$, que cuenta solo las rutas no vacías, y la distancia total $D(x)$ de esas rutas. Una solución es *completa* si $\mathcal{U}(x) = \emptyset$.

Todas las rutas son factibles en todo momento: los operadores de destrucción solo quitan clientes (lo que nunca rompe la factibilidad cuando se cumple la desigualdad triangular) y los de reparación solo ejecutan inserciones aceptadas por el Algoritmo 2. La búsqueda nunca visita soluciones con violaciones de capacidad o de ventanas; la única forma de infactibilidad posible es que queden clientes sin asignar, y la función de costo la penaliza de forma explícita.

El objetivo jerárquico habitual del VRPTW (primero minimizar vehículos, después distancia) se escalariza con un peso fijo por vehículo, al que se antepone una penalización por cada cliente sin asignar:

$$f(x) = 10^6 \cdot |\mathcal{U}(x)| + 10^4 \cdot K(x) + D(x). \tag{4}$$

Mientras los pesos dominen a los términos siguientes (en particular, mientras $D(x) < 10^4$ por cada vehículo de diferencia), comparar f equivale a la comparación lexicográfica $(|\mathcal{U}|, K, D)$: primero colocar a todos los clientes, después usar menos vehículos y por último recorrer menos distancia. Gracias al primer término, f está definida también para soluciones incompletas, que por tanto participan en la búsqueda como cualquier otra; esto es lo que permite la fase de minimización de rutas de la Sección 3.3. La función f es la que usan el criterio de aceptación y la detección de mejoras; las ganancias que alimentan el aprendizaje se miden con el mismo orden lexicográfico, pero en términos relativos (Sección 3.4).

2.5. Solución inicial

La solución inicial se construye con una heurística secuencial del vecino más cercano (Algoritmo 3): cada ruta parte del depósito y agrega repetidamente el cliente no visitado más cercano al último nodo que sea alcanzable, respete la capacidad, llegue dentro de su ventana y permita regresar al depósito antes de b_0. Cuando ningún cliente cumple estas condiciones la ruta se cierra y se abre otra. Es determinista, por lo que ambos algoritmos y todas las semillas parten de la misma solución; su calidad es deliberadamente modesta y toda la mejora posterior corresponde al ALNS.

Algoritmo 3: SoluciónInicial (`Solution::generateInitialSolution`)

Entrada : instancia
Salida : solución x con rutas factibles

mientras *queden clientes sin visitar* **hacer**
  $\pi \leftarrow (0,0)$; $c \leftarrow 0$; $t \leftarrow 0$; $carga \leftarrow 0$
  mientras *verdadero* **hacer**
    $\mathcal{C} \leftarrow \{\, i$ no visitado : alc(c,i), $carga + \mu_i \le Q$, $t_i \le b_i$, $t_i + s_i + d_{i0} \le b_0 \,\}$
      donde $t_i = \max(t + s_c + d_{ci},\, a_i)$
    si $\mathcal{C} \neq \emptyset$ **entonces**
      $i^\star \leftarrow \arg\min_{i \in \mathcal{C}} d_{ci}$
      insertar $i^\star$ al final de π; marcarlo visitado
      $carga \leftarrow carga + \mu_{i^\star}$; $t \leftarrow t_{i^\star}$; $c \leftarrow i^\star$
    si no
      si $c = 0$ **entonces** enviar a $\mathcal{U}$ todos los clientes no visitados ▷ *no caben ni solos*
      salir del bucle
  RecalcularRuta(π); agregar π a $\mathcal{R}(x)$
actualizar $K(x)$ y $D(x)$

3. Bucle de búsqueda común

La búsqueda adaptativa en vecindarios grandes (ALNS) [5, 4] mejora una solución destruyéndola parcialmente y reparándola en cada iteración. El Algoritmo 4 reproduce el método `ALNS::solve`, común a los dos métodos de la tesis. Las llamadas Seleccionar y Aprender son los dos únicos puntos de variación (Secciones 4 y 5).

3.1. Idea general

Una iteración del ALNS hace siempre lo mismo, sea cual sea el mecanismo de selección:

1. **Elegir.** Se elige un operador de destrucción y uno de reparación (Seleccionar). Es la única decisión que toma el mecanismo adaptativo.
2. **Destruir.** Sobre una copia de la solución actual x, el operador de destrucción saca de sus rutas entre $q_{\text{mín}}$ y $q_{\text{máx}}$ clientes (como máximo 60) y los deja sin asignar.
3. **Reparar.** El operador de reparación vuelve a insertar esos clientes en las rutas existentes, uno a uno y siempre de forma factible. El resultado es la candidata x'.
4. **Evaluar y aceptar.** Si x' es mejor que x se acepta; si es peor, se acepta con la probabilidad del recocido simulado, que disminuye a medida que la temperatura baja. Si x' es la mejor solución vista, se guarda en x^*.
5. **Aprender.** El resultado de la iteración (nuevo mejor, mejora, aceptada o rechazada, y cuánto mejoró) se entrega al mecanismo adaptativo (Aprender), que lo usa para cambiar sus preferencias futuras.

El adjetivo *adaptativa* se refiere a ese último paso: los operadores que funcionan bien en la instancia y en la etapa de búsqueda en curso se eligen más a menudo. Los dos métodos de la tesis comparten los pasos 2 a 4 sin ninguna diferencia y solo difieren en cómo implementan los pasos 1 y 5 (Tabla 6).

3.2. Bucle principal

La búsqueda se organiza en *dos fases*, siguiendo el esquema de Ropke y Pisinger [5]: la fase 1 minimiza el número de rutas y la fase 2 minimiza la distancia con la flota obtenida. Ambas ejecutan exactamente el mismo cuerpo de iteración; lo que cambia es la solución de la que se parte y las condiciones de corte (Sección 3.3). En el pseudocódigo, $\bar{x}$ denota la última solución completa conocida y T_{fin} la última iteración, que se fija al comenzar la fase 2.

Algoritmo 4: Bucle ALNS común en dos fases (`ALNS::solve`)

Entrada : solución inicial x_0, presupuesto de iteraciones $T_{\text{máx}}$
Salida : mejor solución encontrada x^*

$\bar{x} \leftarrow x_0$; *fase1* $\leftarrow$ verdadero; $T_{\text{fin}} \leftarrow 2\,T_{\text{máx}}$
$T_0 \leftarrow -\,w\,D(x_0)/\ln 0.5$; $K_{\text{mín}} \leftarrow \lceil \sum_i \mu_i / Q \rceil$
IniciarEtapa(0) ▷ *Algoritmo 5: fija x, x^*, T y los contadores*
$t \leftarrow 1$
mientras $t \le T_{\text{fin}}$ **hacer**
  $x' \leftarrow x$ ▷ *copia de trabajo*
  $(j, k) \leftarrow$ Seleccionar(t) ▷ *destrucción j, reparación k*
  AplicarOperadores(x', j, k) ▷ *Algoritmo 6*
  $b_{\text{ant}} \leftarrow |\mathcal{U}(x^*)|$
  $o.g^{\text{mej}} \leftarrow G(x^*, x')$; $o.g^{\text{act}} \leftarrow G(x, x')$ ▷ *ganancias relativas, ec. (6)*
  $o.nuevoMejor \leftarrow f(x') < f(x^*)$
  $o.mejoró \leftarrow o.nuevoMejor \;\vee\; f(x') < f(x)$
  $o.aceptado \leftarrow o.mejoró \vee$ Aceptar($f(x')$, $f(x)$, T)
  si $o.aceptado$ **entonces** $x \leftarrow x'$
  si $o.nuevoMejor$ **entonces** $x^* \leftarrow x'$
  $estanc \leftarrow 0$ si $o.nuevoMejor$; si no, $estanc + 1$
  si $estanc \ge L_{\text{est}}$ **entonces** ▷ *reinicio por estancamiento*
    $x \leftarrow x^*$; $estanc \leftarrow 0$
    si *fase1* **entonces** $T \leftarrow \rho\,T_0$
  Aprender(t, o)
  $T \leftarrow c\,T$ ▷ *enfriamiento geométrico*
  si *fase1* **entonces** ▷ *control de la fase de rutas*
    $b \leftarrow |\mathcal{U}(x^*)|$
    $sinBanco \leftarrow 0$ si $b < b_{\text{ant}}$; si no, $sinBanco + 1$
    si $b = 0$ **entonces** ▷ *ruta eliminada: se intenta con la siguiente*
      $\bar{x} \leftarrow x^*$; IniciarEtapa(t)
    si no, si $t \ge T_{\text{máx}}$ **o** ($b \ge B$ **y** $sinBanco \ge L_{\text{rut}}$) **entonces** ▷ *se abandona el intento*
      fase1 $\leftarrow$ falso; IniciarEtapa(t)
  $t \leftarrow t + 1$
devolver x^*

Algoritmo 5: IniciarEtapa (función local `startStage` de `ALNS::solve`)

Entrada : iteración t en la que comienza la etapa

$fase1 \leftarrow fase1 \ \wedge \ t < T_{\text{máx}} \ \wedge \ K(\bar{x}) > K_{\text{mín}}$

si $\neg\, fase1$ **entonces** ▷ *comienza la fase 2*

  $T_{\text{fin}} \leftarrow t + T_{\text{máx}}$

  AlIniciarFase2(t) ▷ *aviso a la subclase (`onDistancePhase`)*

$x \leftarrow \bar{x}$

si $fase1$ **entonces** ▷ *fase 1: una ruta menos*

  eliminar de $\mathcal{R}(x)$ las rutas vacías

  $\pi \leftarrow$ ruta de x con menos clientes

  mover todos los clientes de π a $\mathcal{U}(x)$; eliminar π de $\mathcal{R}(x)$

$x^* \leftarrow x$; $T \leftarrow T_0$; $estanc \leftarrow 0$; $sinBanco \leftarrow 0$

Tabla 2: Constantes del bucle común (`alns.cpp`, `operators.cpp`, `solution.cpp`).

Constante	Símbolo	Valor	Función
`UNASSIGNED_COST`	–	10^6	Peso de un cliente sin asignar en f
`VEHICLE_COST`	–	10^4	Peso de un vehículo en f
`TEMP_SCALE`	w	0.2	Empeoramiento relativo aceptado con probabilidad 0.5 en T_0
`COOLING_RATE`	c	0.9995	Factor de enfriamiento por iteración
`STAGNATION_LIMIT`	L_{est}	1500	Iteraciones sin nuevo mejor que disparan el reinicio
`REHEAT_FACTOR`	ρ	0.5	Fracción de T_0 a la que recalienta el reinicio (solo en la fase 1)
`ROUTE_PHASE_BANK`	B	5	Clientes sin asignar a partir de los cuales puede abandonarse un intento de la fase 1
`ROUTE_PHASE_STALL`	L_{rut}	2000	Iteraciones sin reducir los clientes sin asignar que disparan ese abandono
`Q_MIN_RATIO` / `Q_MAX_RATIO`	–	0.10 / 0.40	Fracción mínima y máxima de clientes extraídos
`Q_MIN_CAP` / `Q_MAX_CAP`	–	30 / 60	Topes absolutos de esas dos cotas
`Q_MIN_FLOOR`	–	4	Cota inferior absoluta de q
`NOISE_PROB`	–	0.5	Probabilidad de que la reparación use costos con ruido

3.3. Las dos fases de la búsqueda

Flota fija. Los operadores de reparación *nunca abren rutas nuevas*: insertan los clientes pendientes en las rutas que ya existen en la solución y, si alguno no cabe en ninguna, lo dejan en $\mathcal{U}$ (Sección 7). El número de rutas de una solución solo puede cambiar, por tanto, cuando el bucle común retira una de forma explícita. Esta es la diferencia de fondo con un ALNS de una sola fase, en el que la reparación abre rutas a demanda y la reducción de flota se confía a la función de costo: aquí la flota es un dato de cada etapa y la búsqueda debe encajar a todos los clientes en ella.

Fase 1: minimización de rutas. Se compone de una sucesión de *etapas*. Cada etapa parte de la última solución completa $\bar{x}$, le quita la ruta con menos clientes y deja esos clientes sin asignar (Algoritmo 5). La lista $\mathcal{U}$ actúa como el *banco de clientes* de [5]: los clientes pendientes viajan con la solución de una iteración a la siguiente y cada reparación intenta colocarlos junto con los q recién extraídos. Como f penaliza con 10^6 cada cliente sin asignar, el ALNS ordinario, sin ninguna modificación, trabaja para vaciar el banco. La etapa termina de una de estas tres formas:

- **Éxito.** La mejor solución de la etapa queda completa ($|\mathcal{U}(x^*)| = 0$): se ha eliminado una ruta. Esa solución pasa a ser $\bar{x}$ y comienza otra etapa, con una ruta menos.
- **Abandono por estancamiento.** Quedan al menos $B = 5$ clientes sin asignar en la mejor solución y han pasado $L_{\text{rut}} = 2000$ iteraciones sin que ese número disminuya. Es el criterio de corte de [5]: si el banco es grande y no se reduce, la ruta no es eliminable con un esfuerzo razonable. Con menos de 5 clientes pendientes el intento no se abandona por esta vía y continúa hasta agotar el presupuesto.
- **Presupuesto agotado.** Se alcanza la iteración $T_{\text{máx}}$.

En los dos últimos casos el intento en curso se descarta y la fase 1 termina. También termina, sin iniciar una etapa nueva, cuando $\bar{x}$ alcanza la cota inferior trivial de vehículos

$$K_{\text{mín}} = \Big\lceil \frac{1}{Q} \sum_{i=1}^{n} \mu_i \Big\rceil, \tag{5}$$

ya que con menos rutas no hay capacidad para toda la demanda. Si la solución inicial ya cumple $K(x_0) = K_{\text{mín}}$, la fase 1 no se ejecuta.

Fase 2: minimización de distancia. Parte de $\bar{x}$, la solución completa con menos rutas que encontró la fase 1 (no del intento fallido), y ejecuta siempre $T_{\text{máx}}$ iteraciones: si la fase 1 terminó en la iteración $t_1 \le T_{\text{máx}}$, la búsqueda acaba en $T_{\text{fin}} = t_1 + T_{\text{máx}}$. La flota sigue fija, de modo que el número de vehículos no puede superar el número de rutas de $\bar{x}$; sí puede bajar si una destrucción vacía una ruta y la reparación no vuelve a usarla, lo que f premia con 10^4. Una candidata incompleta cuesta al menos 10^6 más que la solución actual, por lo que en esta fase es rechazada en la práctica (Sección 3.5) y nunca es un nuevo mejor: la solución devuelta es completa.

Reinicio de la búsqueda en cada etapa. Al comenzar cada etapa de la fase 1, y al comenzar la fase 2, se restablecen x, x^*, la temperatura ($T \leftarrow T_0$) y los contadores de estancamiento. En cambio, el estado del mecanismo de selección (los pesos de las ruletas o las tablas Q) *se conserva* entre etapas y entre fases; el único aviso que reciben las subclases es AlIniciarFase2, que el método clásico ignora y el Q-learning aprovecha para pasar a las filas de la fase 2, volver a explorar y reiniciar la escala de sus recompensas (Sección 5.6). Obsérvese que dentro de una etapa de la fase 1 x^* es la mejor solución *de esa etapa*, normalmente incompleta; la mejor solución completa se guarda aparte en $\bar{x}$.

3.4. Ganancias relativas

El resultado o de cada iteración incluye dos ganancias, respecto de la mejor solución y de la actual, calculadas *antes* de actualizar x y x^*. La ganancia de una candidata x' sobre una referencia y es la mejora relativa en el *primer criterio en que difieren*, en el mismo orden que impone f (`relativeGain`):

$$G(y, x') = \begin{cases} \max\Big(0,\ \dfrac{|\mathcal{U}(y)| - |\mathcal{U}(x')|}{|\mathcal{U}(y)|}\Big) & \text{si } |\mathcal{U}(x')| \neq |\mathcal{U}(y)|, \\ \max\Big(0,\ \dfrac{K(y) - K(x')}{K(y)}\Big) & \text{si no, y } K(x') \neq K(y), \\ \max\Big(0,\ \dfrac{D(y) - D(x')}{D(y)}\Big) & \text{en otro caso.} \end{cases} \tag{6}$$

Cada mejora se expresa así en la escala de su propio criterio: colocar un cliente del banco vale $1/|\mathcal{U}|$, ahorrar un vehículo vale $1/K$ y una reducción de distancia vale $\Delta D/D$. Si la ganancia

se midiera sobre f, las mejoras de distancia quedarían diluidas por los términos $10^4 K$ y $10^6|\mathcal{U}|$, que no cambian. Una candidata que empeora el primer criterio en que difiere tiene ganancia 0 aunque mejore los siguientes. Las ganancias solo las utiliza el Q-learning (Sección 5.4); el método clásico usa únicamente las banderas de o.

3.5. Criterio de aceptación

Se usa el criterio de recocido simulado [2]. Una candidata que no empeora la solución actual se acepta siempre; una que la empeora en $\Delta = f(x') - f(x) > 0$ se acepta con probabilidad

$$P(\text{aceptar}) = \exp(-\Delta/T). \tag{7}$$

Obsérvese que una candidata de igual costo ($\Delta = 0$) se acepta pero no se considera mejora, lo que permite desplazamientos laterales por mesetas de f sin que cuenten como éxito para el mecanismo de aprendizaje.

Las candidatas incompletas no reciben un tratamiento especial: se evalúan con f como cualquier otra. En la fase 1 son la norma, y el orden que induce f hace que la búsqueda prefiera, por este orden, menos clientes pendientes, menos vehículos y menos distancia. Dejar un cliente más sin asignar supone $\Delta \geq 10^6$ menos lo que se ahorre en los otros términos, y su probabilidad de aceptación es despreciable a las temperaturas de trabajo; en la práctica, el banco de la solución actual no crece. A igualdad de clientes pendientes, en cambio, el recocido acepta empeoramientos de distancia con normalidad, y es ese movimiento el que reordena las rutas hasta que los clientes del banco encuentran hueco.

3.6. Esquema de temperatura

Temperatura inicial. T_0 se calibra una sola vez, respecto de la distancia de la solución inicial, de forma que un empeoramiento de $w\,D(x_0)$ se acepte con probabilidad 0.5:

$$T_0 = \frac{-\,w\,D(x_0)}{\ln 0.5} = \frac{0.2}{\ln 2}\,D(x_0) \approx 0.2885\,D(x_0). \tag{8}$$

Con esta definición, la probabilidad de aceptar un empeoramiento Δ a temperatura T_0 es $2^{-\Delta/(wD(x_0))}$. Todas las etapas de la fase 1 y la fase 2 arrancan con esta misma T_0. Conviene notar que T_0 escala con la distancia de la instancia, mientras que las penalizaciones de (4) son fijas: la probabilidad de aceptar a T_0 una solución con un vehículo adicional (reutilizando una ruta que había quedado vacía) es $2^{-5\cdot 10^4/D(x_0)}$, que es despreciable cuando $D(x_0)$ es del orden de unos pocos miles y deja de serlo cuando $D(x_0)$ es del orden de decenas de miles. La de aceptar un cliente más sin asignar es $2^{-5\cdot 10^6/D(x_0)}$, despreciable en todos los tamaños de instancia considerados.

Enfriamiento. Tras cada iteración $T \leftarrow c\,T$ con $c = 0.9995$. La temperatura se reduce a la mitad cada $\ln 0.5/\ln c \approx 1386$ iteraciones. En la fase 2, donde no hay recalentamientos, al cabo de sus 25 000 iteraciones vale $c^{25000}\,T_0 \approx 3.7 \cdot 10^{-6}\,T_0$: la fase termina en un descenso prácticamente puro.

Reinicio por estancamiento. Si transcurren $L_{\text{est}} = 1500$ iteraciones consecutivas sin un nuevo mejor, la búsqueda vuelve a la mejor solución conocida ($x \leftarrow x^*$). Este retorno es un mecanismo de *intensificación*, porque descarta la deriva acumulada y retoma el mejor punto conocido, y actúa igual en las dos fases. Lo que ocurre con la temperatura, en cambio, depende de la fase.

En la fase 1 el reinicio además *recalienta*: la temperatura se fija en $\rho\,T_0 = 0.5\,T_0$, lo que vuelve a permitir empeoramientos (*diversificación*). En un periodo prolongado de estancamiento

la temperatura oscila entonces de forma periódica dentro del intervalo

$$[\rho\, c^{L_{\text{est}}}\, T_0,\ \rho\, T_0] \approx [\,0.236\, T_0,\ 0.5\, T_0\,],$$

y nunca llega a congelarse; solo desciende por debajo de esa banda mientras se sigan encontrando nuevos mejores con una separación menor que L_{est} iteraciones. Es el comportamiento buscado en esta fase: el objetivo es vaciar el banco, la distancia del intento no se conserva (la fase 2 parte de $\bar{x}$) y son precisamente los empeoramientos de distancia aceptados los que reordenan las rutas hasta que los clientes pendientes encuentran hueco (Sección 3.5). En la fase 1, «nuevo mejor» es cualquier descenso de f dentro de la etapa, incluidas las mejoras de distancia a igualdad de banco; por eso el abandono de un intento no se decide con este contador sino con uno propio (*sinBanco*), que solo atiende al número de clientes pendientes.

En la fase 2 el reinicio *no modifica la temperatura*, que sigue el enfriamiento geométrico sin interrupciones. Un recalentamiento mantendría la temperatura en la banda anterior, muy por encima de la variación de costo de una iteración: T_0 es proporcional a la distancia total $D(x_0)$, mientras que una iteración solo modifica a lo sumo 60 clientes. A esa temperatura casi cualquier empeoramiento de distancia se acepta y la búsqueda no llega a intensificar. El efecto se agrava con el tamaño de la instancia, porque $D(x_0)$ crece con n y el tamaño del vecindario no (Sección 3.7).

El contador de estancamiento se actualiza *antes* de Aprender, pero el resultado o que recibe el mecanismo de aprendizaje es el de la iteración recién evaluada y no se ve alterado por el reinicio. Del mismo modo, el cambio de etapa o de fase se decide *después* de Aprender, de manera que la iteración que completa una solución en la fase 1 se aprende como cualquier otra.

3.7. Catálogo de operadores y grado de destrucción

El Algoritmo 6 muestra cómo se aplica el par elegido. Hay 6 operadores de destrucción y 5 de reparación (Tabla 3), que ambos métodos eligen por separado.

Algoritmo 6: AplicarOperadores (`applyOperators`)

Entrada : solución x', índice de destrucción $j \in \{0,\dots,5\}$, índice de reparación $k \in \{0,\dots,4\}$

$q_{\text{mín}} \leftarrow \text{máx}\big(4,\ \text{mín}(30,\ \lfloor 0.10\, n \rfloor)\big)$
$q_{\text{máx}} \leftarrow \text{máx}\big(q_{\text{mín}}+1,\ \text{mín}(60,\ \lfloor 0.40\, n \rfloor)\big)$
$q \sim \text{U}\{q_{\text{mín}},\dots,q_{\text{máx}}\}$
$des_j(x', q)$
ruido $\leftarrow$ verdadero con probabilidad 0.5
$rep_k(x', \textit{ruido})$

Tabla 3: Catálogo de operadores, en el orden de sus índices en el código.

Índice	Operador	Criterio
Destrucción		
0	`randomRemoval`	Clientes al azar
1	`routeRemoval`	Rutas completas al azar
2	`worstRemoval`	Clientes cuyo retiro ahorra más distancia (aleatorizado)
3	`shawRemoval`	Clientes parecidos entre sí en espacio, tiempo y demanda
4	`clusterRemoval`	Uno de los dos grupos geográficos de una ruta, y así en rutas vecinas
5	`timeWindowRemoval`	Clientes con las ventanas de tiempo más estrechas
Reparación (cada una, con o sin ruido en los costos)		
0	`greedyInsertion`	Inserción de menor costo
1	`regret2Insertion`	Mayor arrepentimiento entre las dos mejores posiciones
2	`regret3Insertion`	Mayor arrepentimiento entre las tres mejores rutas
3	`regret4Insertion`	Mayor arrepentimiento entre las cuatro mejores rutas
4	`regretMInsertion`	Mayor arrepentimiento sobre todas las rutas

Grado de destrucción. Se sortea de forma uniforme entre el 10 % y el 40 % de los clientes, con los topes absolutos de Pisinger y Ropke [4]:

$$q \in \big[\text{mín}(0.1\,n,\ 30),\ \text{mín}(0.4\,n,\ 60)\big],$$

es decir, $q \in [10, 40]$ para $n = 100$, $q \in [20, 60]$ para $n = 200$ y $q \in [30, 60]$ para todo $n \geq 300$. En las instancias de 400, 600 y 800 clientes el vecindario tiene, por tanto, el mismo tamaño absoluto y representa una fracción decreciente de la solución (entre el 3.75 % y el 7.5 % para $n = 800$). El tope mantiene acotado el costo de una iteración, que está dominado por la reparación (Sección 7), y es lo que hace asumible un presupuesto de hasta $2\,T_{\text{máx}} = 50\,000$ iteraciones en las instancias grandes. En la fase 1 la reparación debe reinsertar, además de los q clientes extraídos, los que ya estaban en el banco.

Ruido. En cada iteración se decide con probabilidad 0.5, de forma independiente del par elegido, si la reparación trabaja con los costos de inserción exactos o con costos perturbados (Sección 7.3). El ruido no es un operador aparte ni una decisión del mecanismo de aprendizaje: cada operador de reparación se aplica, en promedio, la mitad de las veces en cada modalidad, y el crédito que recibe mezcla ambas.

4. ALNS clásico: selección por ruleta adaptativa

`ALNS_Classic` implementa el mecanismo adaptativo de Ropke y Pisinger [5]. Mantiene dos ruletas *independientes*, una para los 6 operadores de destrucción y otra para los 5 de reparación. Cada ruleta (estructura `Roulette` de `alns.h`) guarda, para cada operador i, un peso w_i (inicialmente 1), un puntaje acumulado S_i y un contador de usos u_i dentro del segmento en curso.

La ruleta tiene tres operaciones, que definen el *ritmo* de aprendizaje que también sigue el Q-learning (Sección 5): `select` elige un operador en cada iteración, `record` acumula el resultado del operador usado y `closeSegment` actualiza las preferencias cada $\ell = 100$ iteraciones (`SEGMENT_SIZE`). En palabras simples: durante 100 iteraciones se juega con los pesos actuales y se anota cómo le fue a cada operador; al terminar el bloque, los pesos se corrigen un 10 % hacia el puntaje medio obtenido.

4.1. Selección

En cada iteración se sortea un operador de cada ruleta con probabilidad proporcional a su peso:

$$P(i) = \frac{w_i}{\sum_l w_l}. \tag{9}$$

Como los dos sorteos son independientes, la probabilidad de un par es el producto $P^-(j)\,P^+(k)$: el método no puede representar que un operador de destrucción funcione mejor con un operador de reparación concreto.

4.2. Asignación de crédito y actualización

Tras evaluar la candidata, ambos operadores del par reciben el mismo puntaje σ, que depende únicamente de la *categoría* del resultado y no de su magnitud:

$$\sigma = \begin{cases} 33 & \text{si la candidata es un nuevo mejor global,} \\ 13 & \text{si mejora a la solución actual,} \\ 9 & \text{si no mejora pero es aceptada,} \\ 0 & \text{si es rechazada.} \end{cases} \tag{10}$$

Los tres valores positivos son los propuestos en [5], asignados aquí en orden monótono con la calidad del resultado. Las categorías se determinan comparando valores de f, por lo que valen igual en las dos fases: en la fase 1, colocar un cliente del banco es una mejora (y normalmente un nuevo mejor de la etapa), y una candidata que deja más clientes sin asignar que la solución actual es, en la práctica, un rechazo. Los pesos no cambian en cada iteración sino al cierre de cada segmento de $\ell = 100$ iteraciones, con una media móvil exponencial sobre el puntaje medio del segmento:

$$w_i \leftarrow \max\Big(w_{\min},\ \lambda\, w_i + (1-\lambda)\,\frac{S_i}{u_i}\Big) \quad \text{si } u_i > 0, \qquad \lambda = 0.9, \quad w_{\min} = 0.01. \tag{11}$$

Un operador que no se usó en el segmento conserva su peso. El Algoritmo 7 reúne ambas operaciones.

Algoritmo 7: Selección y aprendizaje del ALNS clásico (`ALNS_Classic` y `Roulette`)

procedimiento SELECCIONAR(t)
  $j \leftarrow$ sorteo en la ruleta de destrucción según (9)
  $k \leftarrow$ sorteo en la ruleta de reparación según (9)
  devolver (j, k)
procedimiento APRENDER(t, o)
  $\sigma \leftarrow$ puntaje de o según (10)
  $S^-_j \leftarrow S^-_j + \sigma$; $u^-_j \leftarrow u^-_j + 1$
  $S^+_k \leftarrow S^+_k + \sigma$; $u^+_k \leftarrow u^+_k + 1$
  si t mód $100 = 0$ **entonces** ▷ *cierre de segmento*
    para cada *ruleta y cada operador i de la ruleta* **hacer**
      si $u_i > 0$ **entonces** $w_i \leftarrow \max\big(w_{\min},\ \lambda w_i + (1-\lambda) S_i/u_i\big)$
      $S_i \leftarrow 0$; $u_i \leftarrow 0$

4.3. Propiedades relevantes

- **Normalización por uso.** El peso se actualiza con el puntaje *medio* S_i/u_i, por lo que un operador elegido con frecuencia no se refuerza solo por haber sido usado más veces.
- **Inercia.** Con $\lambda = 0.9$ cada segmento aporta el 10 % del peso nuevo. El puntaje medio está acotado en $[0, 33]$, de modo que los pesos convergen hacia ese rango de forma gradual: partiendo de $w_i = 1$, hacen falta varios segmentos (varios cientos de iteraciones) para que la distribución se aleje de la uniforme. Los segmentos se cuentan sobre el índice global de iteración, sin distinguir etapas ni fases: en las 25 000 iteraciones de la fase 2 hay 250 actualizaciones de pesos, a las que se suman hasta otras 250 en la fase 1.
- **Continuidad entre fases.** El método no reacciona al cambio de fase (`onDistancePhase` conserva su implementación vacía): los pesos aprendidos mientras se eliminaban rutas son el punto de partida de la fase de distancia, y se readaptan al nuevo objetivo solo a través de la media móvil (11).
- **Exploración permanente.** La selección es proporcional, no voraz, y $w_{\min}$ impide que un peso se anule. Todos los operadores conservan una probabilidad no nula durante toda la búsqueda, y la distribución se adapta a la fase en curso (por ejemplo, los operadores que aceptan empeoramientos puntúan más cuando la temperatura es alta).
- **Crédito ordinal.** Una mejora mínima de distancia y la colocación de un cliente del banco reciben el mismo puntaje si pertenecen a la misma categoría; el método mide con qué frecuencia un operador tiene éxito, no cuánto mejora.

- **Sin contexto.** La distribución no depende del estado de la búsqueda más que a través de los pesos acumulados.

5. ALNS con Q-learning

`ALNS_QLearning` sustituye las dos ruletas del método clásico por dos tablas de Q-learning [10, 9]. El diseño es deliberadamente *simétrico* al del método clásico: se conservan las dos decisiones separadas (una para la destrucción y otra para la reparación), el mismo ritmo de aprendizaje (se elige en cada iteración y se aprende al cerrar cada segmento de $\ell = 100$ iteraciones) y la misma forma de acumular el resultado de cada operador dentro del segmento. En el código, la estructura `QTable` tiene los mismos tres métodos que `Roulette` (Sección 4): `select`, `record` y `closeSegment`.

5.1. La idea en pocas palabras

Lo que cambia respecto de la ruleta son tres cosas:

1. **Qué se premia.** La ruleta premia la *categoría* del resultado (33/13/9/0). El Q-learning premia *cuánto* mejoró la candidata, medido en relación con las mejoras que se vienen obteniendo en la fase (Sección 5.4). Una candidata que no mejora recibe 0, aunque sea aceptada.
2. **Cómo se elige.** La ruleta sortea en proporción a los pesos. El Q-learning es ε-voraz: con probabilidad $1 - \varepsilon$ elige el operador de mayor valor y con probabilidad ε uno al azar. ε empieza en 1 en cada fase y decae hasta 0.1 (Sección 5.3).
3. **Con qué contexto.** La ruleta tiene un único juego de pesos. El Q-learning tiene una fila de valores por *estado*: en qué fase está la búsqueda y si el segmento anterior encontró una nueva mejor solución (Sección 5.2). Así, la preferencia por un operador puede ser distinta mientras se eliminan rutas que mientras se acorta la distancia, o cuando la búsqueda progresa que cuando está estancada.

Todo lo demás (solución inicial, operadores, aceptación, temperatura, fases y semilla) es idéntico en los dos métodos (Sección 1).

Tabla 4: Parámetros del Q-learning. Los cinco primeros son campos de `SolverParams` (`Utils/params.h`) y pueden modificarse desde la línea de comandos con la sintaxis `clave=valor`; los cuatro últimos son constantes del código.

Parámetro	Símbolo	Valor	Función
`alpha`	α	0.1	Tasa de aprendizaje de la tabla Q
`gamma`	γ	0.8	Factor de descuento
`epsilon_min`	$\varepsilon_{\text{mín}}$	0.1	Exploración residual: fracción de elecciones al azar tras el decaimiento
`learning_loop`	ϕ_{exp}	0.05	Fracción de $T_{\text{máx}}$ con exploración pura al inicio de cada fase
`eta`	η	0.8	Peso de la ganancia sobre la mejor solución frente a la ganancia sobre la actual
`EXPLOIT_FROM`	ϕ_{expl}	0.15	Fracción de $T_{\text{máx}}$ en la que ε alcanza $\varepsilon_{\text{mín}}$
`REWARD_CAP`	$r_{\text{máx}}$	2	Tope de la recompensa normalizada
`BASELINE_DECAY`	β	0.9	Inercia de la media móvil de mejoras
`Roulette::SEGMENT_SIZE`	ℓ	100	Longitud del segmento (compartida con el método clásico)

5.2. Estados, acciones y tablas

- **Acciones.** Hay dos agentes que comparten estado y recompensa: uno elige entre los 6 operadores de destrucción y otro entre los 5 de reparación. Cada uno tiene su tabla, $Q^- \in \mathbb{R}^{4\times 6}$ y $Q^+ \in \mathbb{R}^{4\times 5}$, inicializadas en cero. En total se estiman 44 valores, frente a los 11 pesos de la ruleta.
- **Estados.** Cuatro estados, codificados como

$$z = 2\,\mathit{fase} + b, \qquad \mathit{fase} \in \{0,1\}, \quad b \in \{0,1\}, \tag{12}$$

  donde $\mathit{fase} = 0$ es la minimización de rutas, $\mathit{fase} = 1$ la de distancia, y $b = 1$ si el segmento anterior produjo al menos un nuevo mejor (`found_best`). El estado inicial es $z = 0$.
- **El estado solo cambia al cerrar un segmento.** Durante las 100 iteraciones de un segmento el estado es fijo; las dos tablas se consultan siempre en la misma fila.

Como la fase forma parte del estado, las filas $z \in \{0,1\}$ solo se usan en la fase 1 y las filas $z \in \{2,3\}$ solo en la fase 2. En la práctica, cada fase aprende su propia preferencia de operadores partiendo de cero, y lo aprendido mientras se eliminaban rutas no condiciona la elección mientras se acorta la distancia. La única conexión entre ambas es el término de arrastre $\gamma \max Q$ de la última actualización de la fase 1 (Sección 5.6).

5.3. Selección: política ε-voraz por fases

En cada iteración, cada agente elige su operador de forma independiente con la misma ε (Algoritmo 8): con probabilidad ε un operador uniformemente al azar y, en otro caso, el de mayor $Q(z,\cdot)$. Los empates (tolerancia 10^{-12}) se resuelven al azar, lo que importa al principio, cuando todos los valores son cero.

La exploración sigue un calendario que se reinicia al comenzar cada fase. Sea $t' = t - t_{\text{ini}} - 1$ el número de iteraciones transcurridas en la fase en curso ($t_{\text{ini}} = 0$ en la fase 1 y la iteración de cambio en la fase 2) y sean

$$t_{\text{exp}} = \phi_{\text{exp}}\, T_{\text{máx}} = 1250, \qquad t_{\text{expl}} = \max\!\big(t_{\text{exp}} + 1,\ \phi_{\text{expl}}\, T_{\text{máx}}\big) = 3750.$$

Entonces

$$\varepsilon(t') = \begin{cases} 1 & t' < t_{\text{exp}}, \\ \max\!\Big(\varepsilon_{\text{mín}},\ \varepsilon_{\text{mín}}^{(t'-t_{\text{exp}})/(t_{\text{expl}}-t_{\text{exp}})}\Big) & t' \geq t_{\text{exp}}. \end{cases} \tag{13}$$

Es decir, cada fase tiene tres tramos:

1. *Exploración pura* (las primeras 1250 iteraciones): elecciones uniformes. Cada operador de destrucción se prueba unas 200 veces y cada reparación unas 250; la tabla se actualiza con normalidad en este tramo.
2. *Transición* (de la iteración 1250 a la 3750 de la fase): ε decae de forma exponencial desde 1 hasta 0.1.
3. *Explotación* (desde la iteración 3750 de la fase): $\varepsilon = 0.1$; nueve de cada diez elecciones son voraces.

Los umbrales se miden sobre $T_{\text{máx}}$ y no sobre la duración real de la fase, que en la fase 1 es variable: si se midieran sobre ella, una fase 1 corta quedaría casi entera al azar y una larga apenas exploraría. En la fase 2, que dura exactamente $T_{\text{máx}} = 25\,000$ iteraciones, el 85 % transcurre en explotación. Una fase 1 que termina antes de 1250 iteraciones transcurre completa con elección uniforme.

Algoritmo 8: Selección ε-voraz (`ALNS_QLearning::selectOperators` y `QTable::select`)

procedimiento SELECCIONAR(t)
  $\varepsilon \leftarrow \varepsilon(t - t_{\text{ini}} - 1)$ según (13)
  $j \leftarrow$ ELEGIREN(Q^-, z, ε); $k \leftarrow$ ELEGIREN(Q^+, z, ε)
  devolver (j, k)
procedimiento ELEGIREN(Q, z, ε)
  si $\mathrm{U}(0,1) < \varepsilon$ **entonces devolver** una acción uniformemente al azar ▷ *exploración*
  $Q^\star \leftarrow \max_a Q(z,a)$
  devolver un elemento al azar de $\{a : Q(z,a) \geq Q^\star - 10^{-12}\}$ ▷ *explotación*

5.4. Recompensa: mejora relativa normalizada

La recompensa de una iteración se construye a partir de las dos ganancias relativas que calcula el bucle común (Sección 3.4), ambas medidas antes de actualizar x y x^*: $g^{\text{mej}} = G(x^*, x')$ respecto de la mejor solución y $g^{\text{act}} = G(x, x')$ respecto de la actual. Se combinan en una mejora

$$I = \eta\, g^{\text{mej}} + (1-\eta)\, g^{\text{act}}, \qquad \eta = 0.8, \tag{14}$$

que da más peso a superar la mejor solución que a mejorar solo la actual. Si $I = 0$ (la candidata no mejora la solución actual en su primer criterio distinto, aunque sea aceptada) la recompensa es 0.

Si $I > 0$, la mejora se divide por una media móvil $\bar{I}$ de las mejoras positivas observadas en la fase, con un tope, siguiendo la normalización de Kou et al. [3] (`ALNS_QLearning::reward`):

$$r = \begin{cases} 0 & I = 0, \\ 1 & I > 0 \text{ y no hay aún media } (\bar{I} = 0), \\ \min\!\left(I/\bar{I},\ r_{\max}\right) & I > 0 \text{ y } \bar{I} > 0, \end{cases} \tag{15}$$

con $r_{\max} = 2$. Después de calcular r, y solo si $I > 0$, se actualiza la media:

$$\bar{I} \leftarrow \begin{cases} I & \text{si } \bar{I} = 0, \\ \beta\, \bar{I} + (1-\beta)\, I & \text{en otro caso,} \end{cases} \qquad \beta = 0.9. \tag{16}$$

Así, cada mejora se compara con las anteriores, y solo con mejoras positivas.

Por qué normalizar. Las ganancias de (6) tienen escalas muy distintas: colocar un cliente del banco vale $1/|\mathcal{U}|$ (del orden de 10^{-1}), mientras que una mejora de distancia vale $\Delta D/D$ (del orden de 10^{-4} a 10^{-3} y cada vez menor a medida que avanza la búsqueda). Con la recompensa sin normalizar, la tabla quedaría dominada por los primeros éxitos y las mejoras tardías, que son las difíciles, apenas la moverían. El cociente $I/\bar{I}$ expresa cada mejora como «cuántas veces mejor que lo habitual en este momento», una escala que no depende del tamaño de la instancia ni de la etapa de la búsqueda: $r \approx 1$ es una mejora típica, $r < 1$ una mejora pequeña y $r = 2$ una mejora al menos el doble de la habitual. El tope evita que una mejora excepcional (por ejemplo, vaciar el banco en la fase 1 tras una racha de mejoras de distancia) sature la tabla. La media $\bar{I}$ se reinicia al comenzar la fase 2 (Sección 5.6) y no entre etapas de la fase 1.

Acumulación por segmento. Dentro de un segmento, cada agente suma la recompensa de la iteración al operador que eligió y cuenta un uso más (`QTable::record`). Los dos agentes reciben la misma recompensa, igual que en la ruleta ambos operadores reciben el mismo puntaje.

5.5. Actualización de la tabla al cerrar el segmento

Cada $\ell = 100$ iteraciones (cuando t mód $100 = 0$) se cierra el segmento. El siguiente estado es $z' = 2\,\mathit{fase} + b$, con $b = 1$ si en el segmento hubo algún nuevo mejor. Para cada operador a usado $u_a > 0$ veces en el segmento, con recompensa acumulada S_a, se aplica la regla de Q-learning con la recompensa *media* del segmento:

$$Q(z,a) \leftarrow Q(z,a) + \alpha\Big[\frac{S_a}{u_a} + \gamma \max_{a'} Q(z',a') - Q(z,a)\Big], \qquad \alpha = 0.1, \quad \gamma = 0.8. \tag{17}$$

Los operadores no usados conservan su valor. Después se ponen a cero los acumuladores, el estado pasa a ser z' y b vuelve a falso. El Algoritmo 9 resume el paso de aprendizaje.

Algoritmo 9: Aprendizaje por Q-learning (`learn` y `QTable::closeSegment`)

procedimiento APRENDER(t, o)
  $r \leftarrow$ RECOMPENSA($\eta\, o.g^{\text{mej}} + (1-\eta)\, o.g^{\text{act}}$) ▷ *ec.* (15)
  $S^-_j \leftarrow S^-_j + r$; $u^-_j \leftarrow u^-_j + 1$
  $S^+_k \leftarrow S^+_k + r$; $u^+_k \leftarrow u^+_k + 1$
  $b \leftarrow b \ \vee\ o.\mathit{nuevoMejor}$
  si t mód $100 = 0$ **entonces** CERRARSEGMENTO($2\,\mathit{fase} + b$) ▷ *cierre de segmento*
procedimiento CERRARSEGMENTO(z')
  para cada *tabla* $Q \in \{Q^-, Q^+\}$ **hacer**
    $m \leftarrow \max_{a'} Q(z',a')$
    para cada *acción* a *de la tabla* **hacer**
      si $u_a > 0$ **entonces** $Q(z,a) \leftarrow Q(z,a) + \alpha\big[S_a/u_a + \gamma\, m - Q(z,a)\big]$
      $S_a \leftarrow 0$; $u_a \leftarrow 0$
  $z \leftarrow z'$; $b \leftarrow$ falso

Relación con la ruleta. Con $\gamma = 0$, la regla (17) es exactamente la media móvil de la ruleta (11) con $\lambda = 1 - \alpha = 0.9$, aplicada a la recompensa media del segmento en lugar del puntaje medio. Ambos métodos tienen, por tanto, la misma inercia: cada segmento aporta el 10 % del valor nuevo. Las diferencias efectivas son la señal (magnitud normalizada frente a categoría), la regla de elección (voraz frente a proporcional), la fila según el estado y el término de arrastre $\gamma \max Q$, que valora un operador también por la situación a la que conduce (por ejemplo, a un estado $b = 1$ en el que se viene progresando).

Escala de los valores. Como $0 \le S_a/u_a \le r_{\max}$, los valores quedan en $[0,\ r_{\max}/(1-\gamma)] = [0, 10]$. Para un operador que se usa en todos los segmentos de un mismo estado, Q tiende a $\bar{r}_a + \gamma \max Q$; la diferencia entre dos operadores de la misma fila la determina, por tanto, su recompensa media por uso, que combina con qué frecuencia mejoran y cuánto.

5.6. Cambio de fase

Al comenzar la fase 2 el objetivo efectivo cambia: deja de consistir en colocar clientes pendientes y pasa a ser acortar una solución completa. El bucle común avisa a la subclase con ALINICIARFASE2 (Algoritmo 5), y el Q-learning responde como indica el Algoritmo 10.

Algoritmo 10: Inicio de la fase 2 en el Q-learning (`ALNS_QLearning::onDistancePhase`)

Entrada : iteración t en la que comienza la fase 2
CERRARSEGMENTO(2) ▷ *cierra el segmento en curso; pasa al estado* $z = 2$
$\mathit{fase} \leftarrow 1$; $t_{\text{ini}} \leftarrow t$ ▷ *reinicia el calendario de* ε
$\bar{I} \leftarrow 0$ ▷ *olvida la escala de mejoras de la fase 1*

- **Cierre del segmento parcial.** Lo acumulado desde el último cierre se aprende con la transición a $z' = 2$, el primer estado de la fase 2. Los segmentos siguientes se cierran, como siempre, en las iteraciones múltiplo de 100, de modo que el primer segmento de la fase 2 puede ser más corto.
- **Nuevo calendario de exploración.** La fase 2 vuelve a empezar con 1250 iteraciones uniformes y el decaimiento de (13), de modo que todos los operadores se evalúan con el nuevo objetivo.
- **Nueva escala de recompensas.** Las mejoras de la fase 1 (clientes colocados, vehículos eliminados) son órdenes de magnitud mayores que las de distancia; si $\bar{I}$ se conservara, las primeras mejoras de la fase 2 recibirían recompensas casi nulas.
- **La tabla se conserva.** No hace falta borrarla: por (12), la fase 2 usa filas que la fase 1 no tocó.

Las etapas sucesivas de la fase 1 comparten objetivo y no reinician nada: la tabla, el estado, $\bar{I}$ y ε continúan de una etapa a la siguiente.

5.7. Propiedades relevantes

- **Mismo ritmo que la ruleta.** Elegir en cada iteración y aprender cada 100 iteraciones con la recompensa media del segmento hace que cada actualización se base en decenas de usos de cada operador y no en un resultado aislado, que en el ALNS es muy ruidoso (depende del grado q, del ruido y de la temperatura).
- **Solo se premia la mejora.** A diferencia del método clásico, una candidata aceptada que no mejora recibe 0, igual que una rechazada. El agente no refuerza operadores que solo generan soluciones aceptables a temperatura alta.
- **Crédito proporcional a la magnitud.** Dos operadores con la misma tasa de éxito reciben valores distintos si uno produce mejoras mayores; con la ruleta, ambos recibirían el mismo puntaje.
- **Explotación concentrada con exploración residual.** Tras la transición, el 90 % de las elecciones de cada agente recae en el operador mejor valorado de su fila; el 10 % restante mantiene en uso al resto del catálogo y permite que la preferencia cambie si otro operador empieza a rendir más. La ruleta, en cambio, reparte siempre en proporción a los pesos.
- **Contexto.** El estado distingue «la búsqueda progresa» ($b = 1$) de «la búsqueda está estancada» ($b = 0$) y separa por completo las dos fases.
- **Costo despreciable.** Elegir y actualizar cuesta $\mathcal{O}(6 + 5)$ por iteración y por cierre de segmento, frente al costo de la reparación (Sección 7).

6. Operadores de destrucción

Los seis operadores tienen la firma $des(x, q)$: extraen aproximadamente q clientes de las rutas de x y los añaden a $\mathcal{U}(x)$, que en la fase 1 puede contener ya clientes pendientes de iteraciones anteriores (el banco de la Sección 3.3). Ninguno elimina rutas: una ruta que se queda sin clientes permanece en la solución como ruta vacía. Todos terminan actualizando $K(x)$ y $D(x)$. Como salvaguarda, los bucles de las extracciones aleatoria, de rutas, de peores y de Shaw se detienen si $|\mathcal{U}|$ alcanza $n - 1$.

Se denomina *ruta activa* a la que tiene al menos un cliente, y *costo de retiro* de un cliente π_k al ahorro de distancia que produce quitarlo de su ruta:

$$\mathrm{cr}(\pi_k) = d_{\pi_{k-1}\pi_k} + d_{\pi_k\pi_{k+1}} - d_{\pi_{k-1}\pi_{k+1}} \ \geq 0. \tag{18}$$

6.1. Extracción aleatoria

Algoritmo 11: ExtracciónAleatoria (`randomRemoval`)

Entrada : solución x, grado q

mientras *se hayan extraído menos de q clientes* **y** *existan rutas activas* **hacer**
  elegir una ruta activa π uniformemente al azar
  elegir una posición $k \sim \mathrm{U}\{1, \dots, L-2\}$ de π
  mover π_k a $\mathcal{U}$; RecalcularRuta(π)

El sorteo es en dos etapas (primero la ruta, luego el cliente), por lo que no es uniforme sobre los clientes: un cliente de una ruta con $L-2$ clientes se elige con probabilidad $1/(m_{\mathrm{act}}(L-2))$, donde m_{act} es el número de rutas activas, y los clientes de rutas cortas son más propensos a salir. Es el operador de diversificación pura y su costo es $\mathcal{O}\big(q\,(m+\bar{L})\big)$.

6.2. Extracción de rutas

Algoritmo 12: ExtracciónDeRutas (`routeRemoval`)

Entrada : solución x, grado q

mientras *se hayan extraído menos de q clientes* **y** *existan rutas activas* **hacer**
  elegir una ruta activa π uniformemente al azar
  mover todos los clientes de π a $\mathcal{U}$; $\pi \leftarrow (0,0)$

Vacía rutas enteras hasta alcanzar q, por lo que puede superar q en hasta una ruta. La ruta vaciada permanece en la solución como ruta vacía: no cuenta como vehículo, pero queda disponible para la reparación. El operador obliga a redistribuir por completo los clientes de varios vehículos. Como la reparación no abre rutas, es además el principal mecanismo por el que la fase 2 puede terminar con menos vehículos que los heredados de la fase 1: basta con que los clientes de una ruta vaciada se recoloquen en las demás.

6.3. Extracción de los peores

Extrae clientes con costo de retiro alto, es decir, los que están «mal colocados». La elección se aleatoriza como en [5]: con la lista $\mathcal{L}$ de clientes ordenada por costo de retiro decreciente, se toma el elemento de índice

$$\iota = \lfloor y^{p}\,|\mathcal{L}| \rfloor, \qquad y \sim \mathrm{U}(0,1), \quad p = 3. \tag{19}$$

Como $P(\iota < k) = (k/|\mathcal{L}|)^{1/p}$, con $p = 3$ el 10 % superior de la lista se elige con probabilidad 0.46 y el 1 % superior con probabilidad 0.22: hay un sesgo fuerte hacia los peores sin llegar a ser determinista.

Optimización: lista ordenada incremental. La formulación directa recalcula y ordena los costos de todos los clientes antes de cada extracción, con costo $\mathcal{O}(q\,n \log n)$. Pero retirar un cliente solo cambia el costo de retiro de sus dos vecinos en la ruta. El Algoritmo 13 ordena una sola vez y después mantiene la lista: el cliente extraído y sus vecinos se localizan por búsqueda binaria, usando el orden total (costo decreciente, identificador creciente), y se reubican. Un mapa auxiliar cliente → ruta evita buscar al cliente en toda la solución.

Algoritmo 13: ExtracciónDePeores (`worstRemoval`)

Entrada : solución x, grado q

construir $rutaDe[\cdot]$ (cliente → índice de ruta)
calcular $\mathrm{cr}(i)$ para todo cliente en ruta según (18)
$\mathcal{L} \leftarrow$ clientes ordenados por (cr decreciente, identificador creciente)
mientras *se hayan extraído menos de* q *clientes* **y** $\mathcal{L} \neq \emptyset$ **hacer**
  $y \sim \mathrm{U}(0,1)$; $\iota \leftarrow \min\big(\lfloor y^p |\mathcal{L}| \rfloor,\ |\mathcal{L}| - 1\big)$
  si $\mathcal{L}[\iota]$ *empata en costo con* $\mathcal{L}[\iota-1]$ **o** *con* $\mathcal{L}[\iota+1]$ **entonces**
    $i \leftarrow$ ReferenciaPeores(x, ι) ▷ *ordenamiento completo original*
  si no $i \leftarrow \mathcal{L}[\iota]$
  $\pi \leftarrow$ ruta $rutaDe[i]$; $k \leftarrow$ posición de i en π
  quitar i de $\mathcal{L}$ por búsqueda binaria; $rutaDe[i] \leftarrow -1$
  mover i a $\mathcal{U}$; RecalcularRuta(π)
  para cada *vecino* $v \in \{\pi_{k-1}, \pi_k\}$ *con* $v \neq 0$ **hacer**
    $c' \leftarrow \mathrm{cr}(v)$ en la ruta modificada
    si $c' \neq \mathrm{cr}(v)$ *anterior* **entonces** reubicar v en $\mathcal{L}$ por búsqueda binaria con costo c'

Equivalencia con la versión de referencia. Cuando el elemento elegido empata en costo con alguno de sus vecinos en la lista, el orden entre ellos depende del algoritmo de ordenamiento; en ese caso el operador delega la elección en el procedimiento original (`worstByFullSort`), que reconstruye y ordena la lista completa. Cuando no hay empate, el elemento de índice ι es único y coincide en ambas versiones. De este modo la versión optimizada elige siempre el mismo cliente que la original y, para una misma semilla, la trayectoria de búsqueda es idéntica: la optimización reduce el tiempo de cómputo sin alterar los resultados. El mismo principio se aplica en la extracción de Shaw y en los operadores de reparación.

El costo pasa de $\mathcal{O}(q\, n \log n)$ a $\mathcal{O}(n \log n)$ para el ordenamiento inicial más, por extracción, tres búsquedas binarias, el desplazamiento contiguo de elementos de la lista y un recálculo de ruta $\mathcal{O}(\bar{L})$.

6.4. Extracción de Shaw

Propuesta por Shaw [7], extrae clientes *parecidos* entre sí, con la idea de que son intercambiables y la reparación podrá recolocarlos de otra manera. La afinidad entre dos clientes combina distancia, diferencia de apertura de ventana y diferencia de demanda (menor valor significa más afinidad):

$$\mathrm{Rel}(i,j) = 9\, d_{ij} + 3\, |a_i - a_j| + 2\, |\mu_i - \mu_j|. \tag{20}$$

Los pesos $(9, 3, 2)$ son los de [5]; los términos no se normalizan, por lo que su influencia relativa depende de las escalas de la instancia. El primer cliente se elige al azar (ruta y posición). Después, en cada paso, se toma como base un cliente al azar entre los ya extraídos y se extrae uno de los que siguen en ruta, elegido entre los más afines a la base con el mismo sesgo (19) ($p = 3$).

Optimización: orden de afinidad precalculado. $\mathrm{Rel}(i,j)$ depende solo de la instancia, no de la solución. Por ello el orden de afinidad de cada cliente respecto de todos los demás puede calcularse una única vez. La primera invocación del operador construye la tabla $orden[b]$ (`Instance::shaw_order`), que contiene para cada cliente b los otros $n-1$ clientes ordenados por (afinidad creciente, identificador creciente). El costo es $\mathcal{O}(n^2 \log n)$ una sola vez por ejecución y la memoria $n(n-1)$ enteros (unos 2.6 MB para $n = 800$); la tabla pertenece a la instancia y la comparten todas las soluciones.

A partir de ahí no se ordena nada: el Algoritmo 14 recorre la fila de la base saltando los clientes que ya no están en ruta hasta alcanzar el rango deseado. Como el sesgo concentra ι en valores pequeños, el recorrido suele ser corto.

Algoritmo 14: ExtracciónDeShaw (`shawRemoval`)

Entrada : solución x, grado q

elegir al azar una ruta activa y un cliente i_0 de ella; mover i_0 a $\mathcal{U}$
$\mathcal{E} \leftarrow \{i_0\}$ ▷ *clientes extraídos por este operador*
construir $rutaDe[\cdot]$; $enRuta \leftarrow$ número de clientes en ruta
mientras $|\mathcal{E}| < q$ **y** $enRuta > 0$ **hacer**
  $b \leftarrow$ elemento al azar de $\mathcal{E}$
  $y \sim \mathrm{U}(0,1)$; $\iota \leftarrow \min\big(\lfloor y^p \, enRuta \rfloor,\ enRuta - 1\big)$
  recorrer $orden[b]$ saltando los clientes con $rutaDe = -1$ hasta obtener los de rango $\iota - 1$, ι e $\iota + 1$
  $i \leftarrow$ cliente de rango ι
  si Rel(b, i) *empata con la del rango* $\iota - 1$ **o** *la del rango* $\iota + 1$ **entonces**
    $i \leftarrow$ ReferenciaShaw(x, b, ι) ▷ *ordenamiento completo original*
  mover i a $\mathcal{U}$ y a $\mathcal{E}$; RecalcularRuta(ruta de i)
  $rutaDe[i] \leftarrow -1$; $enRuta \leftarrow enRuta - 1$

El costo por extracción pasa de $\mathcal{O}(n \log n)$ (calcular y ordenar las afinidades de todos los clientes en ruta) a un recorrido lineal en el peor caso y típicamente mucho más corto. El tratamiento de empates es el mismo que en la Sección 6.3 y garantiza la equivalencia con la versión de referencia.

6.5. Extracción de grupos

Es la extracción de *clusters* de Pisinger y Ropke [4]. Parte de la observación de que, al extraer clientes parecidos de una ruta, la reparación tiende a devolverlos a ella si el resto de la ruta sigue cerca. El operador divide los clientes de una ruta en dos grupos geográficos y extrae uno *entero*, y repite el proceso en rutas vecinas hasta alcanzar q (Algoritmo 15).

Los dos grupos se obtienen con el algoritmo de Kruskal sobre el grafo completo de los clientes de la ruta, con las distancias d_{ij} como pesos, *detenido cuando quedan dos componentes conexas*: las aristas se recorren de menor a mayor y cada una fusiona las componentes de sus extremos si son distintas (estructura de conjuntos disjuntos con compresión de caminos). El resultado equivale a quitar la arista más larga del árbol de expansión mínima, es decir, a separar la ruta por su mayor salto espacial.

Algoritmo 15: ExtracciónDeGrupos (`clusterRemoval`)

Entrada : solución x, grado q

$\pi \leftarrow$ ruta activa elegida uniformemente al azar; $\mathcal{E} \leftarrow \emptyset$ ▷ *extraídos por este operador*
mientras *exista* π **hacer**
  $\mathcal{C} \leftarrow$ clientes de π
  ordenar los pares $\{a, b\} \subseteq \mathcal{C}$ por d_{ab} creciente
  fusionar componentes en ese orden hasta que queden 2 ▷ *Kruskal truncado*
  $\mathcal{G} \leftarrow$ componente de un cliente de $\mathcal{C}$ elegido uniformemente al azar
  mover los clientes de $\mathcal{G}$ a $\mathcal{U}$ y a $\mathcal{E}$; RecalcularRuta(π)
  si $|\mathcal{E}| \geq q$ **entonces salir del bucle**
  $s \leftarrow$ elemento al azar de $\mathcal{E}$
  $\pi \leftarrow$ ruta, distinta de la actual, que contiene al cliente en ruta más cercano a s (ninguna si no quedan clientes en otras rutas)

Conviene destacar tres detalles:

- **Elección del grupo.** El grupo se elige sorteando un cliente de la ruta, no una de las dos componentes, por lo que la probabilidad de extraer un grupo es proporcional a su tamaño. Si la ruta tiene uno o dos clientes, cada componente es un único cliente; con un solo cliente,

la ruta queda vacía.

- **Encadenamiento.** La siguiente ruta es la del cliente más cercano a uno de los ya extraídos, lo que concentra la destrucción en una región del plano: los grupos extraídos son vecinos entre sí y pueden intercambiarse entre rutas adyacentes. Solo se excluye la ruta recién tratada, de modo que el proceso puede volver más adelante a una ruta ya dividida y dividir lo que quedó de ella.
- **Grado efectivo.** El operador extrae grupos completos, así que puede superar q en hasta un grupo.

Frente a la extracción de Shaw, que elige cliente a cliente por afinidad con sesgo aleatorio, aquí la unidad extraída es un bloque espacial compacto y contiguo de una ruta. El costo por ruta tratada es $\mathcal{O}(\bar{L}^2 \log \bar{L})$ para ordenar los pares, más $\mathcal{O}(n)$ para localizar la ruta siguiente.

6.6. Extracción por ventana de tiempo

Algoritmo 16: ExtracciónPorVentana (`timeWindowRemoval`)

Entrada : solución x, grado q

$\mathcal{C} \leftarrow$ clientes en ruta ordenados por anchura de ventana $b_i - a_i$ creciente
$\mathcal{D} \leftarrow$ los mín$(q, |\mathcal{C}|)$ primeros de $\mathcal{C}$
ordenar $\mathcal{D}$ por (ruta, posición) decreciente ▷ *mantiene válidos los índices al borrar*
para cada $i \in \mathcal{D}$ **hacer**
  mover i a $\mathcal{U}$; RecalcularRuta(ruta de i)

Es un operador específico del VRPTW. Los clientes con ventanas estrechas son los más difíciles de ubicar y los que condicionan la estructura temporal de las rutas; al extraerlos todos a la vez, la reparación puede redecidir su asignación de forma conjunta en lugar de heredar su posición. Es el único operador de destrucción determinista: para una solución y un q dados extrae siempre el mismo conjunto, de modo que su variabilidad proviene de q, del operador de reparación con que se empareje y del ruido de este. Su costo es $\mathcal{O}(n \log n)$.

7. Operadores de reparación

Los cinco operadores tienen la firma $rep(x, ruido)$ y reinsertan los clientes de $\mathcal{U}(x)$ uno a uno. Todos son heurísticas constructivas de inserción: en cada paso eligen un cliente y una posición según su criterio, ejecutan la inserción y repiten. Son instancias de una misma plantilla (Sección 7.2) y comparten las siguientes reglas.

- **Rutas admisibles.** Una ruta admite al cliente u si $carga + \mu_u \leq Q$. Las rutas vacías que haya dejado la destrucción son rutas admisibles como cualquier otra.
- **Costo de inserción.** Es el incremento de distancia Δ del Algoritmo 2, exacto o perturbado con ruido (Sección 7.3). Insertar en una ruta vacía cuesta $2\, d_{0u}$ y *no* incluye la penalización de 10^4 por vehículo: la reparación razona solo en distancia. El control del número de vehículos recae en la estructura en dos fases de la búsqueda y en la función de costo, a través del criterio de aceptación.
- **Flota fija.** La reparación no abre rutas nuevas. Si ningún cliente pendiente tiene una inserción factible en las rutas existentes, termina y deja esos clientes en $\mathcal{U}$. La candidata resultante es incompleta y el bucle común la evalúa con la penalización de 10^6 por cliente sin asignar (Sección 3.3).

7.1. Formulación directa y su costo

Sea $U = |\mathcal{U}|$ el número de clientes por insertar: $U = q$ tras una destrucción en la fase 2, y q más los clientes del banco en la fase 1. La formulación directa evalúa en cada paso todas las posiciones de todas las rutas para todos los clientes pendientes: $\mathcal{O}(U\,n)$ evaluaciones por paso y $\mathcal{O}(U^2 n)$ en total. Aun con $q \le 60$, esto supone del orden de 10^6 evaluaciones de inserción por iteración para $n = 800$, demasiado para un presupuesto de decenas de miles de iteraciones por corrida.

7.2. Inserción con caché por ruta

La observación clave es que una inserción modifica *una sola ruta*. Las evaluaciones de todos los demás pares (cliente, ruta) siguen siendo válidas. El Algoritmo 17 (`cachedInsertion`) mantiene una matriz $M[u][r]$ con un resumen de la ruta r para el cliente pendiente u:

$$M[u][r] = (\mathit{mejor},\ \mathit{segundo},\ \mathit{pos}),$$

el menor y el segundo menor costo de inserción factible de u en r y la posición del primero (∞ si no existen). Calcular una entrada cuesta $\mathcal{O}(L_r)$ gracias a la evaluación en tiempo constante. Tras cada inserción en la ruta $r^\star$ solo se recalcula la columna $r^\star$.

Sobre cada fila se define un *agregador* $G[u]$ que la resume en lo que el criterio necesita: la ruta de destino y una *clave* de prioridad. Los cinco operadores difieren únicamente en el agregador (Sección 7.4).

Algoritmo 17: InserciónConCaché$\langle G\rangle$ (`cachedInsertion<Agg>`)

Entrada : solución x con $\mathcal{U} \neq \emptyset$; bandera *ruido*; tipo de agregador G

$\nu \leftarrow 0.025\, d_{\text{máx}}$ si *ruido*; si no, $\nu \leftarrow 0$ ▷ *amplitud del ruido*

para cada $u \in \mathcal{U}$ **hacer**

  para cada *ruta* r **hacer** $M[u][r] \leftarrow$ Entrada$(u,\, r,\, \nu)$

  $G[u]$.reconstruir$(M[u])$

mientras $\mathcal{U} \neq \emptyset$ **hacer**

  $u^\star \leftarrow \arg\max\{G[u].\mathit{clave} :\ u \in \mathcal{U},\ G[u] \text{ válido}\}$ ▷ *primer máximo*

  si *no existe* $u^\star$ **entonces salir del bucle** ▷ *flota fija: los restantes quedan sin asignar*

  $r^\star \leftarrow G[u^\star].\mathit{ruta}$

  insertar $u^\star$ en $r^\star$ en la posición $M[u^\star][r^\star].\mathit{pos}$; RecalcularRuta$(r^\star)$

  quitar $u^\star$ de $\mathcal{U}$, de M y de G ▷ *en* $\mathcal{O}(1)$*, intercambiando con el último*

  para cada $u \in \mathcal{U}$ **hacer** ▷ *solo cambió la columna* $r^\star$

    $e_{\text{ant}} \leftarrow M[u][r^\star]$; $M[u][r^\star] \leftarrow$ Entrada$(u,\, r^\star,\, \nu)$

    si $M[u][r^\star] \neq e_{\text{ant}}$ **entonces** $G[u]$.actualizar$(M[u],\ r^\star,\ e_{\text{ant}})$

actualizar $K(x)$ y $D(x)$

Algoritmo 18: ENTRADA (`computeEntry`)

Entrada : cliente u, ruta r con secuencia π, amplitud de ruido ν
Salida : $e = (\mathit{mejor},\ \mathit{segundo},\ \mathit{pos})$

$e \leftarrow (\infty, \infty, -)$
si $\mathit{carga}_r + \mu_u > Q$ **entonces devolver** e
para $k \leftarrow 0$ **hasta** $L - 2$ **hacer**
  $(\mathit{factible}, \Delta) \leftarrow$ EVALUARINSERCIÓN$(u,\ \pi,\ k)$
  si $\neg\, \mathit{factible}$ **entonces continuar**
  si $\nu > 0$ **entonces** $\Delta \leftarrow \max\big(0,\ \Delta + \xi\big)$ con $\xi \sim \mathrm{U}(-\nu, \nu)$
  si $\Delta < e.\mathit{mejor}$ **entonces**
    $e.\mathit{segundo} \leftarrow e.\mathit{mejor}$; $e.\mathit{mejor} \leftarrow \Delta$; $e.\mathit{pos} \leftarrow k + 1$
  si no, si $\Delta < e.\mathit{segundo}$ **entonces**
    $e.\mathit{segundo} \leftarrow \Delta$
devolver e

Costo. La inicialización cuesta $\mathcal{O}(U\,n)$ evaluaciones. Cada paso posterior recalcula una columna, $\mathcal{O}(U\,\bar{L})$, y elige el máximo en $\mathcal{O}(U)$. El total es

$$\mathcal{O}\big(U\,n + U^2\,\bar{L}\big) \quad \text{frente a} \quad \mathcal{O}\big(U^2\,n\big),$$

una reducción por un factor del orden del número de rutas $m = n/\bar{L}$ en el término dominante. A esto se suma la actualización de los agregadores, que se analiza a continuación.

7.3. Ruido en los costos de inserción

Cuando la bandera *ruido* está activa (lo está en la mitad de las iteraciones, Sección 3.7), cada costo de inserción factible se perturba con un término aditivo, como en [5]:

$$\tilde{\Delta} = \max\big(0,\ \Delta + \xi\big), \qquad \xi \sim \mathrm{U}(-\nu, \nu), \qquad \nu = 0.025\, d_{\max}, \tag{21}$$

donde $d_{\max}$ es la mayor distancia entre dos nodos de la instancia y 0.025 es la constante `NOISE_RATE`. La amplitud es absoluta, proporcional a la escala geográfica de la instancia y no al costo de cada inserción: altera el orden entre inserciones cuya diferencia es menor que 2ν y respeta las decisiones claras, de modo que dos reparaciones de la misma solución parcial pueden producir resultados distintos. Es el mecanismo de diversificación de la reparación, y se aplica por igual a los cinco operadores.

El ruido se sortea al calcular cada entrada (Algoritmo 18), con un valor independiente por posición, y actúa *antes* de seleccionar el mejor y el segundo mejor costo de la ruta. En consecuencia afecta a las tres decisiones de la reparación: en qué posición de una ruta entraría un cliente, a qué ruta va y en qué orden se insertan los clientes. La entrada perturbada se conserva en la caché mientras su ruta no cambie y se vuelve a sortear cuando la columna se recalcula. La factibilidad no se perturba: una inserción elegida con ruido es siempre factible, y la distancia de la solución se calcula con los costos reales.

7.4. Agregadores y actualización perezosa

Reconstruir un agregador exige recorrer su fila, $\mathcal{O}(m)$. Hacerlo para todos los clientes en cada paso añadiría $\mathcal{O}(U^2 m)$. Cada agregador implementa por ello una operación ACTUALIZAR que, conocida la entrada anterior y la nueva de la única columna modificada, decide en $\mathcal{O}(1)$ si el resumen puede haber cambiado y solo en ese caso reconstruye. Además, ACTUALIZAR ni siquiera se invoca cuando la entrada recalculada es idéntica a la anterior. Las condiciones de salto están escritas de modo que el resultado coincide siempre con el de una reconstrucción completa, incluidos los empates.

Inserción voraz (`GreedyAgg`). Resume la fila en el menor costo y su ruta:

$$c_u = \min_r M[u][r].\mathit{mejor}, \qquad \mathit{clave} = -c_u.$$

Se inserta primero el cliente cuya mejor inserción es la más barata. Actualización: si la columna modificada es la ruta óptima actual, se reconstruye; si es otra, basta comparar su nuevo costo con c_u (y, en caso de empate, preferir la ruta de menor índice, que es lo que elegiría la reconstrucción). Es el criterio más miope: tiende a agotar pronto las inserciones baratas y a dejar para el final a los clientes difíciles.

Arrepentimiento 2 (`Regret2Agg`). Considera los dos menores costos de inserción del cliente sobre *todas las posiciones de todas las rutas*, $c_u^{(1)} \le c_u^{(2)}$ (pueden pertenecer a la misma ruta; por eso cada entrada de M guarda también el segundo mejor costo de la ruta):

$$\mathit{clave} = c_u^{(2)} - c_u^{(1)}, \qquad \mathit{clave} = 10^9 \text{ si solo existe una inserción factible.}$$

Se inserta primero el cliente que más perdería si no obtuviera su mejor posición; un cliente con una única posición factible tiene prioridad absoluta. Actualización: si la columna modificada no es la ruta óptima y tanto su mejor costo anterior como el nuevo superan $c_u^{(2)}$, ni $c_u^{(1)}$ ni $c_u^{(2)}$ pueden cambiar y se omite la reconstrucción.

Arrepentimiento k sobre rutas (`RegretAgg<K>`). Los tres operadores restantes comparten un agregador genérico, parametrizado por k. Los valores usados son $k = 3$ (`regret3Insertion`), $k = 4$ (`regret4Insertion`) y $k = m$, el número de rutas de la solución (`regretMInsertion`, que en el código corresponde a `K = 0`). Es el arrepentimiento de [5]: considera el mejor costo *por ruta* y ordena las rutas factibles, $c_u^{[1]} \le c_u^{[2]} \le c_u^{[3]} \le \dots$ Con $h = \min(k, \text{número de rutas factibles})$,

$$\mathit{clave} = \sum_{l=2}^{h} (c_u^{[l]} - c_u^{[1]}) + 10^4\,(k - h). \tag{22}$$

El primer término mide cuánto se perdería en total si el cliente tuviera que conformarse con su segunda, tercera, …, k-ésima mejor ruta. El segundo prioriza a los clientes con menos de k rutas factibles, con una penalización de 10^4 por cada ruta que falta. La ruta de destino es la de menor costo (la de menor índice en caso de empate).

El agregador conserva, además de la clave y la ruta, el k-ésimo menor costo $c_u^{[k]}$ (o ∞ si hay menos de k rutas factibles). Actualización: si el mejor costo de la columna modificada no cambió, o si tanto el anterior como el nuevo superan $c_u^{[k]}$, la ruta estaba y sigue estando fuera de las k mejores y la clave no cambia; en otro caso se reconstruye, con un ordenamiento $\mathcal{O}(m \log m)$ sobre un búfer reutilizado para evitar reservas de memoria. Para $k = 3$ y $k = 4$ el salto se aplica siempre que la columna modificada no esté entre las k mejores rutas del cliente. Para $k = m$ ninguna ruta factible queda fuera, de modo que el agregador se reconstruye siempre que cambia el mejor costo de la columna: el arrepentimiento sobre todas las rutas es el operador de reparación más costoso.

Diferencias entre los arrepentimientos. El arrepentimiento 2 se mide sobre posiciones y los demás sobre rutas. El primero es sensible a la diferencia entre las dos mejores posiciones aunque estén en la misma ruta, y por tanto prioriza a los clientes cuya colocación *exacta* importa; los segundos miden cuántas rutas alternativas tiene realmente el cliente y priorizan a los que dependen de pocas rutas. Entre estos, k regula el horizonte de anticipación: con $k = 3$ o $k = 4$ solo cuentan las mejores alternativas, mientras que con $k = m$ el término $10^4(m - h)$ suele dominar la clave y el operador inserta primero, en esencia, a los clientes que caben en *menos* rutas, usando la suma de diferencias para desempatar. Frente a la inserción voraz, todos anticipan qué clientes

quedarán sin alternativas, lo que es especialmente valioso con ventanas de tiempo estrechas y, sobre todo, con flota fija: en la fase 1 un cliente que se queda sin ruta factible permanece en el banco (Sección 3.3).

7.5. Resumen de las optimizaciones

La Tabla 5 reúne las optimizaciones descritas en esta sección y en la anterior. Todas son *exactas*: cambian el costo computacional, no la decisión que toma el operador. En las reparaciones con ruido, la perturbación forma parte de la definición del operador (se sortea una vez por entrada calculada) y la caché no introduce ninguna aproximación adicional.

Tabla 5: Optimizaciones algorítmicas de los operadores. U: clientes por insertar; $\bar{L}$: longitud media de ruta; m: número de rutas.

Componente	Técnica	Costo directo	Costo optimizado
Factibilidad de inserción	Llegadas, esperas y holguras por ruta	$\mathcal{O}(\bar{L})$	$\mathcal{O}(1)$
Arcos imposibles	Matriz de alcanzabilidad precalculada	–	filtro $\mathcal{O}(1)$
Extracción de peores	Lista ordenada incremental; solo se actualizan dos vecinos	$\mathcal{O}(q\,n\log n)$	$\mathcal{O}(n\log n + q\,n)^{\dagger}$
Extracción de Shaw	Orden de afinidad precalculado por instancia	$\mathcal{O}(q\,n\log n)$	$\mathcal{O}(q\,n)^{\ddagger}$
Voraz y arrepentimientos	Caché por (cliente, ruta); solo se recalcula la ruta modificada	$\mathcal{O}(U^2 n)$	$\mathcal{O}(Un + U^2\bar{L})$
Agregadores	Actualización perezosa con prueba $\mathcal{O}(1)$	$\mathcal{O}(m)$ por cliente	$\mathcal{O}(1)$ habitual[§]
Borrado de pendientes	Intercambio con el último elemento	$\mathcal{O}(U)$	$\mathcal{O}(1)$

[†] El término $q\,n$ corresponde a desplazamientos de memoria contigua, con constante muy pequeña.
[‡] Cota de peor caso del recorrido; en la práctica es mucho menor por el sesgo hacia rangos bajos. Se añade un precálculo único de $\mathcal{O}(n^2\log n)$.
[§] Salvo en el arrepentimiento sobre todas las rutas, que reconstruye su agregador en cada cambio.

8. Síntesis: características que determinan el comportamiento

Esta sección recoge, a partir del análisis anterior, las características de diseño que condicionan los resultados de ambos métodos. Se distinguen las que son comunes, y por tanto explican el nivel de calidad que alcanzan los dos, de las que los diferencian.

8.1. Características comunes

1. **Rutas siempre factibles.** Las rutas nunca violan capacidad ni ventanas de tiempo. La única infactibilidad que admite la búsqueda son los clientes sin asignar, que tienen un peso fijo de 10^6 en f; no hay otras penalizaciones que calibrar.
2. **Objetivo jerárquico escalarizado.** Los pesos de 10^6 por cliente sin asignar y de 10^4 por vehículo hacen que colocar un cliente domine sobre cualquier reducción de flota, y esta sobre cualquier mejora de distancia, en la aceptación y en la detección de mejoras.
3. **Reducción de flota en una fase propia.** La reparación no abre rutas, de modo que la flota es fija en cada etapa. La fase 1 elimina rutas una a una (retira la más corta y busca recolocar sus clientes con el ALNS completo) y se detiene al alcanzar la cota inferior de capacidad, al estancarse con 5 o más clientes pendientes durante 2000 iteraciones o al agotar $T_{\text{máx}}$ iteraciones. La fase 2 optimiza la distancia durante otras $T_{\text{máx}}$ iteraciones con

la flota resultante. El número de vehículos obtenido depende, por tanto, de la eficacia con que cada método selecciona operadores *durante la fase 1*.

4. **Vecindarios de tamaño acotado.** Se extrae entre el 10 % y el 40 % de los clientes, con topes de 30 y 60: a partir de 300 clientes el vecindario tiene el mismo tamaño absoluto, lo que mantiene estable el costo por iteración al crecer la instancia.

5. **Catálogo complementario.** Hay operadores de diversificación (extracción aleatoria), de intensificación (extracción de peores, inserción voraz), de estructura (Shaw, grupos, rutas), específicos de las ventanas de tiempo y con anticipación (arrepentimiento 2, 3, 4 y sobre todas las rutas). El ruido en los costos de inserción, activo en la mitad de las iteraciones, diversifica cualquiera de las reparaciones.

6. **Recocido con reinicios.** La temperatura proporcional a la distancia inicial, el enfriamiento geométrico y el retorno a la mejor solución tras 1500 iteraciones sin progreso se aplican en las dos fases. En la fase 1 ese retorno recalienta además a $0.5\,T_0$, lo que evita que la búsqueda se congele mientras intenta vaciar el banco; en la fase 2 no hay recalentamiento y la temperatura desciende sin interrupciones hasta un descenso prácticamente puro. La temperatura vuelve a T_0 al comenzar cada etapa de la fase 1 y al comenzar la fase 2.

7. **Eficiencia por iteración.** La evaluación de inserciones en tiempo constante, las cachés por ruta y las estructuras incrementales de los operadores de destrucción reducen el costo de la reparación en un factor del orden del número de rutas. Al ser optimizaciones exactas, no modifican la trayectoria de búsqueda: lo que aportan es que un presupuesto de hasta 50 000 iteraciones por corrida sea alcanzable en las instancias de mayor tamaño.

8.2. Diferencias entre los dos métodos

Como ambos métodos comparten todo lo anterior, parten de la misma solución inicial con la misma semilla y además aprenden con el mismo ritmo (eligen en cada iteración y actualizan sus preferencias cada 100 iteraciones), cualquier diferencia de resultados se origina en los aspectos de la Tabla 6.

Tabla 6: Comparación de los mecanismos de selección y aprendizaje.

Aspecto	ALNS clásico	ALNS con Q-learning
Unidad de decisión	Destrucción y reparación por separado	Igual: dos tablas, destrucción y reparación
Valores aprendidos	$6 + 5$ pesos	$4 \times 6 + 4 \times 5 = 44$ valores Q
Contexto	Ninguno	Estado: fase y si el segmento anterior encontró un nuevo mejor
Política	Sorteo proporcional a los pesos	ε-voraz; ε de 1 a 0.1 en cada fase
Señal de crédito	Categoría del resultado: 33, 13, 9, 0	Magnitud de la mejora relativa, dividida por la media móvil de mejoras y con tope 2
Aceptación sin mejora	Se premia (9)	Recompensa 0, igual que un rechazo
Ritmo	Elige cada iteración; actualiza cada 100	Igual
Actualización	Media móvil del puntaje medio, $\lambda = 0.9$	Q-learning con la recompensa media, $\alpha = 0.1$ (misma inercia) y $\gamma = 0.8$
Exploración	Permanente y proporcional ($w_{\text{mín}} = 0.01$)	1250 iteraciones uniformes al inicio de cada fase, transición hasta la iteración 3750 y después 10 % al azar
Cambio de fase	Sin reacción: los pesos continúan	Filas propias para la fase 2, nueva exploración y nueva escala de recompensas

De la tabla se desprenden cuatro diferencias de comportamiento que conviene tener presentes al interpretar los resultados experimentales:

- **Qué se entiende por éxito.** El método clásico premia la aceptación, de modo que a temperatura alta refuerza a los operadores que generan soluciones aceptables aunque no mejoren. El Q-learning solo premia la mejora estricta y la pondera por su tamaño respecto de lo que se viene consiguiendo.
- **Sensibilidad a mejoras pequeñas.** Al final de la fase 2 casi todas las mejoras son pequeñas. Para la ruleta todas valen lo mismo (13 o 33); el Q-learning, al dividir por la media reciente, sigue distinguiendo qué operador consigue las mayores dentro de esa escala reducida.
- **Concentración.** La ruleta mantiene en uso todo el catálogo en proporción a los pesos; el Q-learning, tras la exploración inicial, dedica el 90 % de las elecciones al operador mejor valorado de cada estado. El primero es más diverso; el segundo, más intensivo.
- **Adaptación al cambio de objetivo.** Al pasar de eliminar rutas a reducir distancia, el método clásico arrastra los pesos de la fase 1 y los corrige al ritmo de su media móvil; el Q-learning empieza la fase 2 con filas nuevas y vuelve a explorar todos los operadores antes de explotar.

Estas afirmaciones describen el mecanismo, no su efecto medido. Los resultados de la Sección 9 muestran *dónde* se traduce en una diferencia de calidad; para atribuirla a una de estas cuatro diferencias en concreto haría falta registrar la frecuencia de uso de cada operador por fase en ambos métodos, o ablaciones que activen una diferencia a la vez.

9. Resultados experimentales

Esta sección compara los dos métodos en cinco tamaños de instancia: las 56 instancias de Solomon [8] de 100 clientes y las de Gehring y Homberger [1] de 200, 400, 600 y 800 clientes. Todos los resultados proceden de la versión del código descrita en este documento. Los datos crudos están en `Experiments/results/master_solomon_100.csv` y `master_gh<N>.csv`, y los promedios por instancia en `Experiments/summary/`.

9.1. Diseño experimental

- **Instancias.** Seis clases en cada tamaño: C1, C2, R1, R2, RC1 y RC2. La letra indica la distribución geográfica de los clientes (agrupada, aleatoria o mixta) y el dígito el horizonte de planificación: en las clases de tipo 1 las ventanas son estrechas y las rutas cortas, con muchos vehículos; en las de tipo 2 el horizonte es largo y bastan pocas rutas largas. Solomon tiene 9, 8, 12, 11, 8 y 8 instancias por clase; Gehring y Homberger, 10 por clase y tamaño (60 por tamaño).
- **Presupuesto.** $T_{\text{máx}} = 25\,000$ iteraciones en todos los tamaños: entre 25 000 y 50 000 iteraciones por corrida, según lo que consuma la fase 1. Los parámetros del Q-learning son los de la Tabla 4.
- **Repeticiones y semillas pareadas.** 10 corridas por instancia y algoritmo; la corrida k usa la semilla k en los dos algoritmos, de modo que la comparación es pareada. En total, 1120 corridas en Solomon y 1200 en cada tamaño de Gehring y Homberger. Las 5920 corridas terminaron con una solución válida (rutas factibles y ningún cliente sin asignar), según el verificador independiente de `Utils/utils.cpp`.
- **Referencia.** Mejores soluciones conocidas (BKS) publicadas por SINTEF para el objetivo jerárquico, en `Experiments/sintef/`.

- **Ejecución.** Las campañas de Gehring y Homberger se ejecutaron en un clúster (SLURM) con 32 procesos simultáneos por nodo. El tiempo de CPU está afectado por esa concurrencia y solo es orientativo para comparar los dos métodos entre sí.

9.2. Métricas

Para cada instancia y algoritmo se promedian las 10 corridas: número medio de vehículos NV y distancia media TD. Las medias por clase o globales son medias de esas medias por instancia.

- **Flota.** Se cuentan las instancias en las que cada método obtiene menos vehículos medios que el otro y se contrasta la diferencia con la prueba de rangos con signo de Wilcoxon sobre las instancias.
- **Mejora de distancia (IMP).** Solo es comparable con la misma flota: con un vehículo de más la distancia suele bajar sin que la solución sea mejor. Por eso se calcula en las instancias en las que ambos métodos tienen la misma flota media:

$$\mathrm{IMP}_{\mathrm{TD}} = 100\,\frac{\mathrm{TD}_{\mathrm{ALNS}} - \mathrm{TD}_{\mathrm{Q}}}{\mathrm{TD}_{\mathrm{ALNS}}}\ (\%),$$

  positiva si el Q-learning recorre menos. Se reporta su media, cuántas instancias gana cada método y la prueba de Wilcoxon.
- **Brecha auditada respecto del BKS (GAP).** Por la misma razón, la brecha de distancia $100\,(\mathrm{TD} - \mathrm{TD}_{\mathrm{BKS}})/\mathrm{TD}_{\mathrm{BKS}}$ solo se calcula si la flota media no supera la del BKS; para comparar los métodos se promedia sobre las instancias en las que ambos cumplen esa condición.
- **Marcador lexicográfico.** Cada instancia se asigna al método con menor flota media y, a igualdad de flota, al de menor distancia media; si ambas coinciden es un empate. Se reporta como ganadas–empatadas–perdidas desde el punto de vista del Q-learning, con una prueba de signos (binomial) sobre las instancias no empatadas.
- **Nivel de significación.** 0.05. Con 10 instancias por clase, el menor p posible en la prueba de Wilcoxon es 0.002, de modo que las pruebas por clase solo detectan efectos muy consistentes.

9.3. Resumen por tamaño

Las Tablas 7 y 8 resumen los cinco tamaños y la Figura 1 muestra cómo evoluciona la diferencia.

Tabla 7: Resumen global por tamaño: flota, distancia con la misma flota y marcador. NV: vehículos medios por instancia. «Q / A»: instancias en las que gana el Q-learning / el ALNS clásico. IMP en porcentaje (positivo favorece al Q-learning).

	Flota					Distancia, instancias con la misma flota				Marcador	
n	BKS	ALNS	ALNS-Q	Menos NV (Q / A)	p	Inst.	IMP	Gana (Q / A)	p	G–E–P	p
100	7.23	7.30	7.30	4 / 5	0.763	47	0.00	10 / 20	0.213	14–17–25	0.108
200	11.57	11.59	11.58	3 / 0	0.102	57	+0.34	40 / 11	< 0.001	43–6–11	< 0.001
400	23.00	23.20	23.18	7 / 2	0.109	51	+1.11	43 / 6	< 0.001	50–2–8	< 0.001
600	34.42	34.74	34.70	10 / 3	0.041	47	+1.30	41 / 5	< 0.001	51–1–8	< 0.001
800	45.53	46.21	46.18	9 / 6	0.132	45	+1.50	41 / 3	< 0.001	50–1–9	< 0.001

Tabla 8: Resumen global por tamaño: brecha auditada respecto del BKS (en porcentaje, sobre las instancias en que ambos igualan la flota del BKS), comparación por corrida con la misma semilla y tiempo medio de CPU por corrida.

	GAP auditado				Corridas con	Corridas pareadas (Q / A)		CPU (s)	
n	Inst.	ALNS	ALNS-Q	p	flota BKS (A / Q)	Menos vehículos	Menos distancia†	ALNS	ALNS-Q
100	47	0.66	0.66	0.289	522 / 521 de 560	13 / 14	151 / 154	5.5	5.6
200	57	1.99	1.63	< 0.001	584 / 590 de 600	9 / 3	298 / 198	18.0	16.5
400	44	5.21	4.05	< 0.001	490 / 499 de 600	31 / 17	378 / 144	30.2	28.2
600	40	8.91	7.42	< 0.001	449 / 459 de 600	37 / 15	382 / 148	34.9	37.3
800	30	7.31	5.92	< 0.001	321 / 325 de 600	39 / 25	424 / 102	49.9	54.0

† Solo entre los pares de corridas (misma instancia y semilla) que obtienen el mismo número de vehículos; el resto son empates exactos.

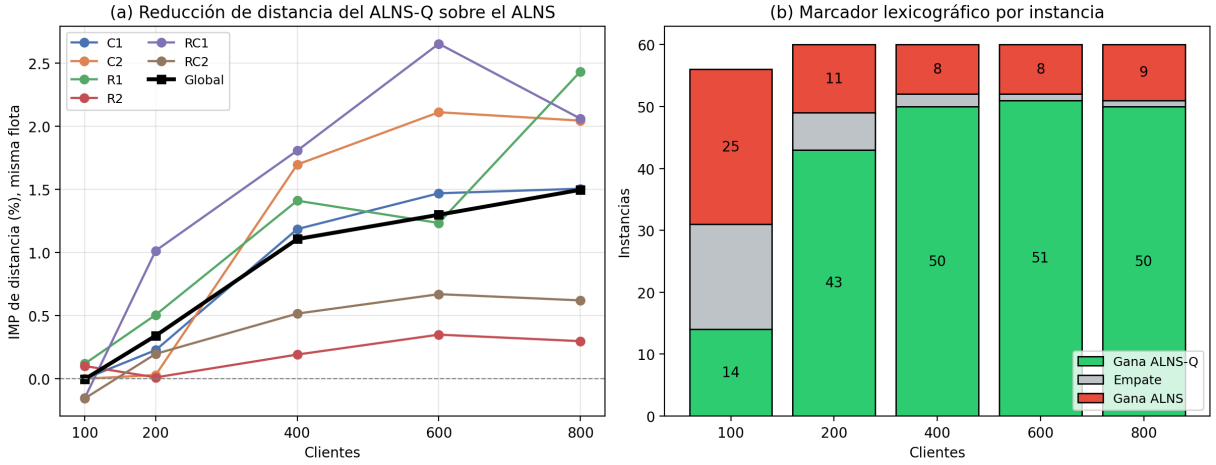


Figura 1: (a) IMP medio de distancia del Q-learning sobre el ALNS clásico, en las instancias con la misma flota media, por clase y global. (b) Marcador lexicográfico por instancia.

9.4. Resultados por tamaño

Las tablas de este apartado usan las mismas columnas: NV / TD medios de cada clase (BKS, ALNS clásico y ALNS-Q); número de instancias con menos flota media para cada método; IMP medio de distancia y, entre paréntesis, número de instancias con la misma flota sobre el que se calcula; GAP auditado medio de cada método y número de instancias en que ambos igualan la flota del BKS; y marcador del Q-learning (ganadas–empatadas–perdidas).

9.4.1. 100 clientes (Solomon)

Tabla 9: Solomon, 100 clientes: resultados por clase.

Clase	BKS NV / TD	ALNS NV / TD	ALNS-Q NV / TD	Menos NV (Q / A)	IMP TD (inst.)	GAP A / Q (inst.)	G–E–P
C1	10.00 / 828.4	10.00 / 828.4	10.00 / 828.4	0 / 0	0.00 (9)	0.00 / 0.00 (9)	0–9–0
C2	3.00 / 589.9	3.00 / 589.9	3.00 / 589.9	0 / 0	0.00 (8)	0.00 / 0.00 (8)	0–8–0
R1	11.92 / 1210.3	12.10 / 1216.8	12.13 / 1213.0	1 / 3	+0.12 (8)	0.63 / 0.51 (8)	4–0–8
R2	2.73 / 951.0	2.78 / 964.1	2.77 / 964.0	1 / 1	+0.10 (9)	1.54 / 1.44 (9)	5–0–6
RC1	11.50 / 1384.2	11.61 / 1387.3	11.61 / 1388.6	1 / 1	−0.15 (6)	0.65 / 0.80 (6)	2–0–6
RC2	3.25 / 1119.2	3.26 / 1135.8	3.25 / 1140.4	1 / 0	−0.16 (7)	1.16 / 1.32 (7)	3–0–5
Global	7.23 / 1021.2	7.30 / 1028.0	7.30 / 1027.9	4 / 5	0.00 (47)	0.66 / 0.66 (47)	14–17–25

Empate. Los dos métodos obtienen la misma flota media (7.30; $p = 0.76$) y la misma brecha auditada (0.66 %). Con la misma flota, la distancia media es igual (IMP = 0.00 %); el ALNS

clásico recorre menos en 20 instancias y el Q-learning en 10, sin significación ($p = 0.21$). El marcador, 14–17–25, favorece algo al clásico, tampoco de forma significativa ($p = 0.11$), y procede de diferencias de una décima de vehículo o de décimas de punto porcentual de distancia. Las 17 instancias de C1 y C2 se resuelven igual, en el BKS, en todas las corridas; en el resto, ambos quedan a menos del 2 % del BKS. Este conjunto está cerca de su techo con 25 000 iteraciones: cualquier mecanismo de selección razonable llega a soluciones equivalentes, y por eso no sirve para predecir el comportamiento en instancias mayores.

9.4.2. 200 clientes

Tabla 10: Gehring y Homberger, 200 clientes: resultados por clase.

Clase	BKS NV / TD	ALNS NV / TD	ALNS-Q NV / TD	Menos NV (Q / A)	IMP TD (inst.)	GAP A / Q (inst.)	G–E–P
C1	18.90 / 2718.4	18.90 / 2743.5	18.90 / 2737.2	0 / 0	+0.23 (10)	0.91 / 0.68 (10)	5–4–1
C2	6.00 / 1831.6	6.00 / 1837.5	6.00 / 1837.0	0 / 0	+0.03 (10)	0.34 / 0.31 (10)	6–2–2
R1	18.20 / 3611.9	18.20 / 3691.3	18.20 / 3672.5	0 / 0	+0.51 (10)	2.22 / 1.70 (10)	9–0–1
R2	4.00 / 2929.4	4.07 / 2949.6	4.05 / 2958.5	1 / 0	+0.01 (9)	1.53 / 1.52 (9)	6–0–4
RC1	18.00 / 3176.2	18.00 / 3325.8	18.00 / 3291.4	0 / 0	+1.01 (10)	4.62 / 3.55 (10)	10–0–0
RC2	4.30 / 2535.9	4.39 / 2585.5	4.35 / 2592.0	2 / 0	+0.20 (8)	2.34 / 2.14 (8)	7–0–3
Global	11.57 / 2800.6	11.59 / 2855.5	11.58 / 2848.1	3 / 0	+0.34 (57)	1.99 / 1.63 (57)	43–6–11

Aparece la ventaja, en distancia. La flota es prácticamente la misma (el Q-learning tiene menos vehículos medios en 3 instancias y el clásico en ninguna; $p = 0.10$), pero con la misma flota el Q-learning recorre menos: IMP medio +0.34 % en 57 instancias, con 40 ganadas y 11 perdidas ($p < 0.001$), y la brecha auditada baja de 1.99 % a 1.63 % ($p < 0.001$). El marcador es 43–6–11 ($p < 0.001$). La ventaja se concentra en RC1 (10–0–0, +1.01 %) y R1 (9–0–1, +0.51 %); en C1, C2 y R2 la diferencia de distancia es pequeña (+0.23, +0.03 y +0.01 %). Por corrida, entre los pares con la misma flota, el Q-learning recorre menos en 298 y el clásico en 198. El Q-learning fue además un 8 % más rápido en tiempo de CPU (16.5 frente a 18.0 s).

9.4.3. 400 clientes

Tabla 11: Gehring y Homberger, 400 clientes: resultados por clase.

Clase	BKS NV / TD	ALNS NV / TD	ALNS-Q NV / TD	Menos NV (Q / A)	IMP TD (inst.)	GAP A / Q (inst.)	G–E–P
C1	37.60 / 7167.6	37.93 / 7414.1	37.88 / 7347.0	3 / 2	+1.18 (5)	2.16 / 0.92 (5)	6–2–2
C2	11.60 / 3939.9	12.04 / 4013.4	12.01 / 3928.4	2 / 0	+1.70 (8)	4.25 / 2.10 (4)	8–0–2
R1	36.40 / 8360.6	36.40 / 8865.3	36.40 / 8740.9	0 / 0	+1.41 (10)	6.02 / 4.51 (10)	10–0–0
R2	8.00 / 6142.7	8.00 / 6379.3	8.00 / 6363.5	0 / 0	+0.19 (10)	3.92 / 3.72 (10)	7–0–3
RC1	36.00 / 7869.4	36.07 / 8474.5	36.03 / 8343.3	1 / 0	+1.81 (9)	8.03 / 6.07 (9)	10–0–0
RC2	8.40 / 5285.2	8.77 / 5406.7	8.75 / 5392.4	1 / 0	+0.52 (9)	4.96 / 4.73 (6)	9–0–1
Global	23.00 / 6460.9	23.20 / 6758.9	23.18 / 6685.9	7 / 2	+1.11 (51)	5.21 / 4.05 (44)	50–2–8

La ventaja se triplica. El IMP medio sube a +1.11 % (51 instancias con la misma flota; 43 ganadas y 6 perdidas; $p < 0.001$) y la brecha auditada baja de 5.21 % a 4.05 % ($p < 0.001$). El marcador es 50–2–8. La flota sigue sin diferencia significativa (23.18 frente a 23.20; 7 instancias con menos flota para el Q-learning y 2 para el clásico; $p = 0.11$), aunque el Q-learning iguala la flota del BKS en 499 corridas frente a 490. R1 y RC1 son 10–0–0 con +1.41 y +1.81 %; la ventaja se extiende a C1 y C2 (+1.18 y +1.70 %) y a RC2 (9–0–1); R2 sigue siendo la clase con menor diferencia (+0.19 %, 7–0–3). El tiempo de CPU es un 7 % menor para el Q-learning (28.2 frente a 30.2 s).

9.4.4. 600 clientes

Tabla 12: Gehring y Homberger, 600 clientes: resultados por clase.

Clase	BKS NV / TD	ALNS NV / TD	ALNS-Q NV / TD	Menos NV (Q / A)	IMP TD (inst.)	GAP A / Q (inst.)	G–E–P
C1	57.20 / 14296.1	57.67 / 14499.5	57.63 / 14264.2	1 / 1	+1.47 (8)	3.76 / 2.00 (6)	8–1–1
C2	17.40 / 7571.3	18.27 / 7876.9	18.10 / 7735.8	7 / 1	+2.11 (2)	– (0)	9–0–1
R1	54.50 / 17807.5	54.53 / 20320.7	54.53 / 20058.0	0 / 0	+1.23 (10)	13.91 / 12.42 (9)	10–0–0
R2	11.00 / 12293.9	11.00 / 12988.1	11.00 / 12951.5	0 / 0	+0.35 (10)	5.74 / 5.36 (10)	7–0–3
RC1	55.00 / 15943.0	55.01 / 17741.0	55.00 / 17286.2	1 / 0	+2.65 (9)	11.21 / 8.24 (9)	10–0–0
RC2	11.40 / 10553.5	11.94 / 11080.7	11.94 / 10988.7	1 / 1	+0.67 (8)	8.39 / 7.51 (6)	7–0–3
Global	34.42 / 13077.6	34.74 / 14084.5	34.70 / 13880.7	10 / 3	+1.30 (47)	8.91 / 7.42 (40)	51–1–8

Único tamaño con diferencia de flota significativa. El Q-learning obtiene menos vehículos medios en 10 instancias y el clásico en 3 ($p = 0.041$), sobre todo en C2 (7 frente a 1; 18.10 frente a 18.27 vehículos de media). Por corrida, con la misma semilla, el Q-learning usa menos vehículos en 37 pares y el clásico en 15. Aun así, la diferencia es pequeña en términos absolutos (34.70 frente a 34.74 vehículos de media). En distancia, el IMP medio es +1.30 % (47 instancias; 41 ganadas y 5 perdidas; $p < 0.001$) y la brecha auditada baja de 8.91 % a 7.42 % ($p < 0.001$). El marcador es 51–1–8. RC1 alcanza la mayor diferencia de todo el estudio (+2.65 %); el +2.11 % de C2 se calcula sobre solo 2 instancias con la misma flota. El tiempo de CPU es un 7 % mayor para el Q-learning (37.3 frente a 34.9 s).

9.4.5. 800 clientes

Tabla 13: Gehring y Homberger, 800 clientes: resultados por clase.

Clase	BKS NV / TD	ALNS NV / TD	ALNS-Q NV / TD	Menos NV (Q / A)	IMP TD (inst.)	GAP A / Q (inst.)	G–E–P
C1	74.90 / 25261.3	75.96 / 26100.1	75.85 / 25765.2	3 / 2	+1.50 (5)	4.84 / 2.86 (4)	7–1–2
C2	23.10 / 11729.4	24.40 / 12459.1	24.35 / 12027.0	4 / 2	+2.04 (4)	0.06 / 0.04 (1)	7–0–3
R1	72.80 / 31033.9	72.80 / 34202.5	72.80 / 33367.6	0 / 0	+2.43 (10)	10.19 / 7.49 (10)	10–0–0
R2	15.00 / 19834.8	15.00 / 21010.5	15.00 / 20943.9	0 / 0	+0.30 (10)	6.04 / 5.72 (10)	8–0–2
RC1	72.00 / 28660.6	73.00 / 31090.6	73.00 / 30448.6	0 / 0	+2.06 (10)	– (0)	10–0–0
RC2	15.40 / 16409.0	16.10 / 17175.8	16.10 / 17085.7	2 / 2	+0.62 (6)	7.51 / 6.81 (5)	8–0–2
Global	45.53 / 22154.8	46.21 / 23673.1	46.18 / 23273.0	9 / 6	+1.50 (45)	7.31 / 5.92 (30)	50–1–9

Mayor ventaja en distancia. El IMP medio llega a +1.50 % (45 instancias con la misma flota; 41 ganadas y 3 perdidas; $p < 0.001$) y la brecha auditada baja de 7.31 % a 5.92 % ($p < 0.001$). Por corrida, en los pares con la misma flota, el Q-learning recorre menos en 424 y el clásico en 102. El marcador es 50–1–9. La flota vuelve a ser equivalente (46.18 frente a 46.21; 9 frente a 6 instancias; $p = 0.13$); en RC1 ambos métodos usan 73 vehículos en todas las corridas, uno más que el BKS, por lo que esa clase no entra en la brecha auditada (esto explica que la brecha media de 800 sea menor que la de 600: se promedia sobre un subconjunto distinto). R1 (+2.43 %) y RC1 (+2.06 %) son 10–0–0. Las clases de horizonte largo son la parte débil: C2 (7–0–3) y RC2 (8–0–2) concentran 5 de las 9 derrotas, y en C2 ambos métodos quedan a más de un vehículo del BKS. El tiempo de CPU es un 8 % mayor para el Q-learning (54.0 frente a 49.9 s).

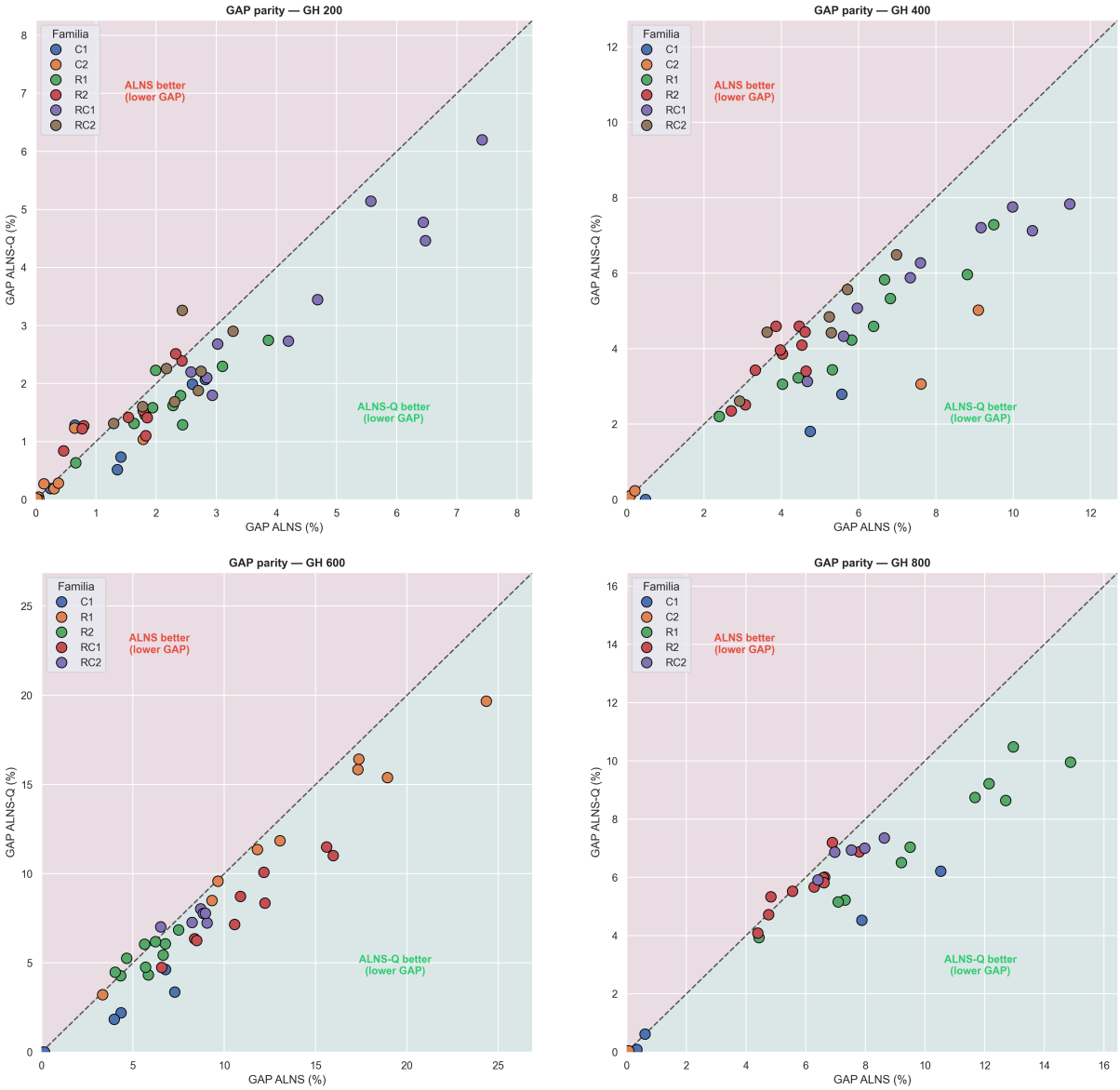


Figura 2: Brecha auditada por instancia, ALNS clásico (abscisas) frente a ALNS-Q (ordenadas), en 200, 400, 600 y 800 clientes. Por debajo de la diagonal el Q-learning es mejor. Solo aparecen las instancias en que ambos igualan la flota del BKS.

9.5. Dónde surge la ventaja de la propuesta

La Tabla 14 cruza clase y tamaño, y resume lo observado en los apartados anteriores.

Tabla 14: IMP medio de distancia (%, instancias con la misma flota) y marcador del Q-learning (G–E–P) por clase y tamaño.

	100		200		400		600		800	
Clase	IMP	G–E–P	IMP	G–E–P	IMP	G–E–P	IMP	G–E–P	IMP	G–E–P
C1	0.00	0–9–0	+0.23	5–4–1	+1.18	6–2–2	+1.47	8–1–1	+1.50	7–1–2
C2	0.00	0–8–0	+0.03	6–2–2	+1.70	8–0–2	+2.11	9–0–1	+2.04	7–0–3
R1	+0.12	4–0–8	+0.51	9–0–1	+1.41	10–0–0	+1.23	10–0–0	+2.43	10–0–0
R2	+0.10	5–0–6	+0.01	6–0–4	+0.19	7–0–3	+0.35	7–0–3	+0.30	8–0–2
RC1	−0.15	2–0–6	+1.01	10–0–0	+1.81	10–0–0	+2.65	10–0–0	+2.06	10–0–0
RC2	−0.16	3–0–5	+0.20	7–0–3	+0.52	9–0–1	+0.67	7–0–3	+0.62	8–0–2
Global	0.00	14–17–25	+0.34	43–6–11	+1.11	50–2–8	+1.30	51–1–8	+1.50	50–1–9

1. **Crece con el tamaño de la instancia.** En 100 clientes no hay diferencia; a partir de 200 el Q-learning es significativamente mejor y el IMP medio crece de forma monótona: 0.00, +0.34, +1.11, +1.30 y +1.50 %. El marcador pasa de 14–17–25 a 43–6–11 en 200 y se estabiliza en torno a 50 de 60 instancias ganadas desde 400.

2. **Está en la distancia, no en la flota.** La flota media de los dos métodos difiere como mucho en 0.04 vehículos por instancia en cada tamaño y solo es significativamente distinta en 600 ($p = 0.041$, concentrada en C2). La ventaja es una reducción de la longitud de las rutas *con el mismo número de vehículos*, que es lo que optimiza la fase 2. No se ha medido por separado cuánto de esa ventaja se gesta en la fase 2 y cuánto en la solución que entrega la fase 1.

3. **Es máxima en las clases de rutas cortas.** R1 y RC1 (ventanas estrechas, muchas rutas cortas: de 18 a 73 vehículos) son 10–0–0 en todos los tamaños desde 400 (RC1 ya desde 200), con IMP entre +1.2 y +2.7 %. C1 y C2 se suman desde 400. Las clases de horizonte largo y pocas rutas, R2 y RC2, muestran la menor ventaja (+0.2 a +0.7 % desde 400) y acumulan 21 de las 36 derrotas entre 200 y 800 clientes; en 800, C2 y RC2 son el punto débil de la propuesta.

4. **Coincide con un presupuesto escaso respecto del tamaño.** El presupuesto es el mismo en todos los tamaños ($T_{\text{máx}} = 25\,000$) y el grado de destrucción está acotado a 60 clientes, de modo que el número de iteraciones de la fase 2 por cliente cae de 250 en $n = 100$ a unas 31 en $n = 800$, y cada iteración modifica una fracción cada vez menor de la solución. Las brechas de ambos métodos respecto del BKS crecen con n (Tabla 8), es decir, hay más margen de mejora. Una explicación coherente con los datos, aunque no probada aquí, es que con pocas iteraciones por cliente elegir bien el operador pesa más, y que en Solomon el presupuesto sobra para que cualquier selección llegue al mismo resultado.

5. **Costo computacional equivalente.** El tiempo de CPU difiere menos de un 10 % en todos los tamaños, a favor del Q-learning en 200 y 400 y en contra en 600 y 800. Como los dos métodos ejecutan los mismos operadores, la diferencia refleja qué operadores elige cada uno (unos son más caros que otros, como el arrepentimiento sobre todas las rutas) y la carga del clúster, no el costo del aprendizaje, que es despreciable.

En resumen, la ventaja de la propuesta no aparece en el banco de pruebas clásico de 100 clientes, donde ambos métodos empatan, sino a partir de 200 clientes, y consiste en rutas entre un 0.3 y un 1.5 % más cortas en promedio con la misma flota, que crecen con el tamaño. Desde 400 clientes la reducción es de 1.2 a 2.7 % en R1 y RC1, donde el Q-learning gana todas las instancias, mientras que en las clases de rutas largas (R2 y RC2) es menor y menos consistente.

Referencias

[1] H. Gehring y J. Homberger. A parallel hybrid evolutionary metaheuristic for the vehicle routing problem with time windows. En *Proceedings of EUROGEN99*, pp. 57–64, 1999.

[2] S. Kirkpatrick, C. D. Gelatt y M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[3] Kou et al. [Completar título y fuente]. 2026.

[4] D. Pisinger y S. Ropke. A general heuristic for vehicle routing problems. *Computers & Operations Research*, 34(8):2403–2435, 2007.

[5] S. Ropke y D. Pisinger. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4):455–472, 2006.

[6] M. W. P. Savelsbergh. The vehicle routing problem with time windows: Minimizing route duration. *ORSA Journal on Computing*, 4(2):146–154, 1992.

[7] P. Shaw. Using constraint programming and local search methods to solve vehicle routing problems. En *Principles and Practice of Constraint Programming – CP98*, LNCS 1520, pp. 417–431. Springer, 1998.

[8] M. M. Solomon. Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2):254–265, 1987.

[9] R. S. Sutton y A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2.ª edición, 2018.

[10] C. J. C. H. Watkins y P. Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.